

Efficient Multi-Party Private Set Union Without Non-Collusion Assumptions

Minglang Dong¹, Cong Zhang², Yujie Bai¹, and Yu Chen¹(✉)

¹ School of Cyber Science and Technology, Shandong University,
Qingdao 266237, China

² Institute for Advanced Study, BNRist, Tsinghua University, Beijing, China
{minglang.dong,baiyujie}@mail.sdu.edu.cn, zhangcong@mail.tsinghua.edu.cn,
yuchen@sdu.edu.cn

Abstract. Multi-party private set union (MPSU) protocol enables m ($m > 2$) parties, each holding a set, to collectively compute the union of their sets without revealing any additional information to other parties. There are two main categories of multi-party private set union (MPSU) protocols: The first category builds on public-key techniques, where existing works require a super-linear number of public-key operations, resulting in poor practical efficiency. The second category builds on oblivious transfer and symmetric-key techniques. The only work in this category, proposed by Liu and Gao (ASIACRYPT 2023), features the best concrete performance among all existing protocols, but still has super-linear computation and communication. Moreover, it does not achieve the standard semi-honest security, as it inherently relies on a non-collusion assumption, which is unlikely to hold in practice.

There remain two significant open problems so far: no MPSU protocol achieves semi-honest security based on oblivious transfer and symmetric-key techniques; no MPSU protocol achieves both linear computation and linear communication complexity. In this work, we resolve both of them.

- We propose the first MPSU protocol based on oblivious transfer and symmetric-key techniques in the standard semi-honest model. This protocol’s online performance is $3.9 - 10.0\times$ faster than Liu and Gao in the LAN setting. Concretely, our protocol requires only 4.4 seconds in online phase for 3 parties with sets of 2^{20} items each.
- We propose the first MPSU protocol achieving linear computation and linear communication complexity, based on public-key operations. This protocol has the best total performance in the WAN settings, due to a factor of $3.0 - 36.5\times$ improvement in terms of overall communication compared to Liu and Gao. Concretely, on slow network (50 Mbps), their protocol takes 6.6 hours to run for 9 parties with sets of 2^{18} items each, whereas ours only takes 47 minutes.

We implement our protocols and conduct extensive experiments to compare the performance of our protocols and the state-of-the-art. To the best of our knowledge, our code is the first correct and secure implementation of MPSU that reports on large-size experiments.

1 Introduction

Over the last decade, there has been growing interest in private set operation (PSO), which consists of private set intersection (PSI), private set union (PSU), and private computing on set intersection (PCSI), etc. Among these functionalities, PSI, especially two-party PSI [50, 38, 46, 18, 47, 55, 52], has made tremendous progress and become highly practical with extremely fast and cryptographically secure implementations. Meanwhile, multi-party PSI [39, 44, 15, 6] is also well-studied. In contrast, the advancement of PSU has been sluggish until recently, several works proposed efficient two-party PSU protocols [40, 29, 34, 58, 19, 35]. However, multi-party PSU has still not been extensively studied. In this work, we focus on PSU in the multi-party setting.

Multi-party private set union (MPSU) enables m ($m > 2$) mutually untrusted parties, each holding a private set of elements, to compute the union of their sets without revealing any additional information. MPSU and its variants have numerous applications, such as information security risk assessment [41], IP blacklist and vulnerability data aggregation [33], joint graph computation [11], distributed network monitoring [37], building block for private DB supporting full join [40], private ID [29], etc.

According to the underlying techniques, existing MPSU protocols can be divided into two categories:³

- **PK-MPSU**: This category is primarily based on public-key techniques, and has been explored in a series of works [37, 27, 57, 28]. A common drawback of these works is that each party has to perform a substantial number of public-key operations, leading to unsatisfactory practical efficiency.
- **SK-MPSU**: This category is primarily based on symmetric-key techniques, and includes only one existing work [42] to date. This work exhibits much better performance than all prior works but fails to achieve standard semi-honest security due to its inherent reliance on a non-collusion assumption, assuming the party who obtains the union (we call it leader hereafter) not to collude with other parties.

Both categories share one common limitation: neither of them includes a protocol achieving linear computation and communication complexity.⁴ Motivated by the above, we raise the following two questions:

Can we construct an MPSU protocol based on oblivious transfer and symmetric-key operations, without any non-collusion assumptions? Can we

³ We only consider the special-purpose solutions for MPSU, excluding those based on circuit-based generic techniques of secure computation, due to their unacceptable performance.

⁴ In the context of MPSU, linear complexity means that the complexity per party scales linearly with the total size of all parties' sets. In this paper, we consider the balanced setting where each party holds sets of equal size, thus linear complexity denotes the complexity per party to scale linearly with both the number of parties m and the set size n . Meanwhile, following current conventions, linear complexity only considers the online phase.

construct an MPSU protocol with both linear computation and linear communication complexity?

1.1 Our Contribution

In this work, we answer the above two questions affirmatively. Our contributions are summarized as follows:

Efficient batch ssPMT. We present a new primitive called batch secret-shared private membership test (batch ssPMT). Compared to multi-query secret-shared private membership test (mq-ssPMT), which is the technical core of the state-of-the-art MPSU protocol [42] (hereafter referred to as LG), batch ssPMT admits a much more efficient construction. Combined with hashing-to-bins, batch ssPMT can replace mq-ssPMT in the existing MPSU frameworks. Looking ahead, batch ssPMT serves as a core building block in our two MPSU protocols, significantly contributing to our speedup to LG.

SK-MPSU in standard semi-honest model. We generalize random oblivious transfer (ROT) into multi-party setting and present a new primitive, multi-party secret-shared random oblivious transfer (mss-ROT). Based on batch ssPMT and mss-ROT, we propose the first SK-MPSU in standard semi-honest model. In addition to enhanced security, our SK-MPSU has superior online / total performance with a $3.9 - 10.0\times / 1.2 - 7.8\times$ improvement in the LAN setting.

PK-MPSU with linear complexity. Based on batch ssPMT and multi-key rerandomizable public-key encryption (MKR-PKE) [28], we propose the first MPSU protocol with both linear computation and communication. Our PK-MPSU has the lowest overall communication costs with a factor of $3.0 - 36.5\times$ improvement compared to LG, making it particularly excel in bandwidth-constrained networks. Along the way, we find that the MPSU protocol of Gao et al. [28] is insecure against arbitrary collusion and give a practical attack to demonstrate that it requires non-collusion assumption.⁵

Figure 1 depicts the technical overview of our new MPSU framework. We will elaborate the details in Section 2.

1.2 Related Works

We review the existing semi-honest MPSU protocols below.

⁵ After we pointed out the security flaw of the GNT protocol [28] with a concrete attack, the authors of [28] contacted us and confirmed our attack. Subsequently, they updated their paper and revised their original protocol (cf. Appendix B in the new version of [28]) to a new one, which is similar to our PK-MPSU. The main difference lies in that they implement our batch ssPMT by invoking multiple instances of ssPMT separately, which renders their new protocol still has superlinear complexities. See also the summary of the relationship in their updated paper.

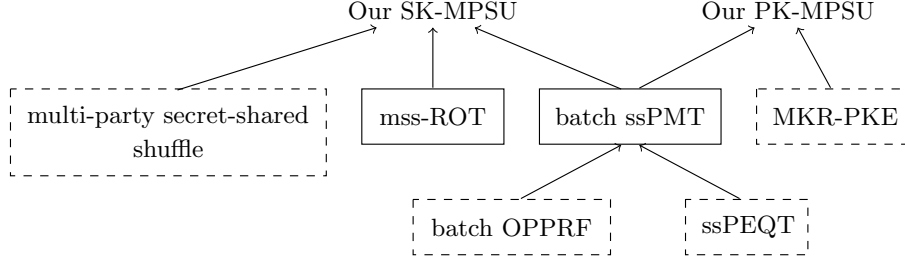


Fig. 1. Technical overview of our MPSU framework. The newly introduced primitives are marked with solid boxes. The existing primitives are marked with dashed boxes.

PK-MPSU. Kissner and Song [37] introduced the first MPSU protocol, based on polynomial representations and additively homomorphic encryption (AHE). This protocol requires a substantial number of AHE operations and high-degree polynomial calculations, so it is completely impractical.

Frikken [27] improved [37] by decreasing the polynomial degree. However, the number of AHE operations remains quadratic in the set size due to the necessity of performing multi-point evaluations on the encrypted polynomials.

Vos et al. [57] proposed an MPSU protocol based on the bit-vector representations, where the parties collectively compute the union by performing the private OR operations on the bit-vectors, using ElGamal encryption. It shows poor concrete efficiency reported by [42] and the leader requires quadratic computation and communication in the number of parties.

Recently, Gao et al. [28] proposed an MPSU protocol, representing the most advanced MPSU in terms of asymptotic complexity. Unfortunately, their protocol turns out to be insecure against arbitrary collusion. We propose a practical attack to show that it is vulnerable to the same colluding attack as LG (see Appendix A for details).

SK-MPSU. Recently, Liu and Gao [42] proposed a practical MPSU protocol based on oblivious transfer and symmetric-key operations. This protocol is several orders of magnitude faster than the prior works. For instance, when computing on datasets of 2^{10} element, it is $109\times$ faster than [57]. However, their protocol is not secure in the standard semi-honest model.

Other Related Works. Blanton et al. [8] proposed an MPSU protocol based on oblivious sorting and generic multi-party computation (MPC). The heavy dependency on general MPC leads to inefficiency.

Table 1 provides a comprehensive theoretical comparison between existing MPSU protocols and our proposed protocols. Leader denotes the participant who obtains the union result. Client refers to the remaining participants.

2 Technical Overview

Protocol	Computation		Communication		Round	Security
	Leader	Client	Leader	Client		
[37]	$m^2 n^3$ pub	$m^2 n^3$ pub	$\lambda m^3 n^2$	$\lambda m^3 n^2$	m	✓
[27]	mn^2 pub	mn^2 pub	λmn	λmn	m	✓
[57]	$lm^2 n$ pub	lmn pub	$\lambda lm^2 n$	λlmn	l	✓
[8]	$\sigma mn \log n + m^2$ sym		$\sigma^2 mn \log n + \sigma m^2$		$\log m$	✓
[28]	$mn(\log n / \log \log n)$ pub		$(\gamma + \lambda)mn(\log n / \log \log n)$		$\log \gamma + m$	✗
[42]	$(T + \gamma' + m)mn$ sym	$(T + \gamma')mn$ sym	$(T + \gamma')mn + lm^2 n$	$(T + \gamma')mn$	$\log(l - \log n) + m$	✗
Our SK-MPSU	$m^2 n$ sym	$m^2 n$ sym	$\gamma mn + lm^2 n$	$(\gamma + l + m)mn$	$\log \gamma + m$	✓
Our PK-MPSU	mn pub	mn pub	$(\gamma + \lambda)mn$	$(\gamma + \lambda)mn$	$\log \gamma + m$	✓

Table 1. Asymptotic communication (bits) and computation costs of MPSU protocols in the semi-honest setting. For the sake of comparison, we omit the Big O notations and simplify the complexity by retaining only the dominant terms. We use ✓ to denote protocols in the standard semi-honest model and ✗ to denote protocols requiring non-collusion assumption. pub: public-key operations; sym: symmetric cryptographic operations. n is the set size. m is the number of parties. λ and σ are computational and statistical security parameter. T is the number of AND gate in a SKE decryption circuit in [42]. l is the bit length of input elements. γ is the output length of OPPRF. γ' is the bit length of SKE ciphertext and $\gamma' \approx \gamma$. In the typical setting, $n \leq 2^{24}$, $m \leq 32$, $\lambda = 128$, $\sigma = 40$, $T \approx 600$, $l \leq 128$, $\gamma \leq 64$.

2.1 LG Revisit

We start by abstracting the high-level idea of LG as a secret-sharing based MPSU framework with two phases: The first phase involves $m - 1$ secret-sharing processes, where each P_j ($2 \leq j \leq m$) somehow secret-shares $Y_j = X_j \setminus (X_1 \cup \dots \cup X_{j-1})$ among all parties. For each element in $X_j \setminus Y_j$, the parties hold a random secret-sharing. Since $\{X_1, Y_2, \dots, Y_m\}$ is a partition of $X_1 \cup \dots \cup X_m$, at the end of these secret-sharing processes, the parties hold secret shares of $(X_1 \cup \dots \cup X_m) \setminus X_1$ in the order of $Y_2, \dots, Y_m$, interspersed with random secret-sharings. The second phase is to reconstruct all these secrets to P_1 . To achieve the reconstruction, a straightforward approach is insecure because P_1 can identify which party each reconstructed element originates from, based on the position of its secret-sharing. The solution is to let the parties invoke multi-party secret-shared shuffle to randomly permute and re-share all secret-sharings. This ensures that any coalition of $m - 1$ parties has no knowledge of the permutation, thereby concealing the correspondence between secret shares and individual difference sets $Y_2, \dots, Y_m$. Each P_j sends its shuffled shares to P_1 , who then reconstructs $(X_1 \cup \dots \cup X_m) \setminus X_1$ and obtains the union by appending the elements in X_1 .

LG utilizes two ingredients to realize the above secret-sharing processes: (1) The secret-shared private membership test (ssPMT) [21, 59], where the sender $\mathcal{S}$ inputs a set X , and the receiver $\mathcal{R}$ inputs an element y . If $y \in X$, $\mathcal{S}$ and $\mathcal{R}$ receive secret shares of 1, otherwise secret shares of 0. Liu and Gao proposed multi-query ssPMT (mq-ssPMT), which supports the receiver querying multiple elements' memberships of the sender's set simultaneously. Namely, $\mathcal{S}$ inputs X , and $\mathcal{R}$ inputs $y_1, \dots, y_n$. $\mathcal{S}$ and $\mathcal{R}$ receive secret shares of a bit vector of size n , where if $y_i \in X$, the i th bit is 1, otherwise 0. (2) A two-choice-bit version of random oblivious transfer (ROT), where the sender $\mathcal{S}$ and the receiver $\mathcal{R}$ each holds a choice bit e_0, e_1 . $\mathcal{S}$ receives two random messages r_0, r_1 . If $e_0 \oplus e_1 = 0$, $\mathcal{R}$ receives

r_0 , otherwise r_1 . However, their construction has two main drawbacks: First, the only existing instantiation of mq-ssPMT heavily relies on expensive general MPC machinery, which becomes the bottleneck of the entire protocol. Second, the resulting first phase introduces a non-collusion assumption to ensure security. In the following sections, we address these two drawbacks by presenting two new primitives, batch ssPMT and mss-ROT, and making use of them to redesign the secret-sharing processes and eliminate the non-collusion assumption.

2.2 Efficient Batch ssPMT

The batch ssPMT is essentially the batched version of ssPMT: The sender $\mathcal{S}$ inputs n disjoint sets $X_1, \dots, X_n$, and the receiver $\mathcal{R}$ inputs n elements $y_1, \dots, y_n$. $\mathcal{S}$ and $\mathcal{R}$ receive secret shares of a bit vector of size n , where the i th bit is 1 if $y_i \in X_i$, and 0 otherwise. Compared to mq-ssPMT, batch ssPMT enables the testing of element membership across multiple distinct sets rather than within a single common set. Their relationship is analogous to that between batched oblivious pseudorandom function (batch OPRF) [38] and multi-point oblivious pseudorandom function (multi-point OPRF) [46, 18]. The batching nature of batch ssPMT admits a much more efficient construction, resulting in a superior alternative to mq-ssPMT in context of MPSU, by coupling with hashing-to-bins.

The alternative to mq-ssPMT works as follows: First, $\mathcal{S}$ and $\mathcal{R}$ preprocess inputs through hashing-to-bins technique. $\mathcal{R}$ uses hash functions h_1, h_2, h_3 to assign $y_1, \dots, y_n$ to B bins via Cuckoo hashing [45], ensuring that each bin accommodates at most one element. At the same time, $\mathcal{S}$ assigns each $x \in X$ to all bins determined by $h_1(x), h_2(x), h_3(x)$. Then they invoke B instances of ssPMT, where in the j th instance, $\mathcal{S}$ inputs the subset $X_j \in X$ containing all elements mapped into its bin j , while $\mathcal{R}$ inputs the sole element mapped into its bin j . If some y_i is mapped to bin j and $y_i \in X$, then $\mathcal{S}$ certainly maps y_i to bin j (and other bins) as well, ensuring $y_i \in X_j$. Therefore, for each bin j of $\mathcal{R}$, if the inside element y_i is in X , $y_i \in X_j$, $\mathcal{S}$ and $\mathcal{R}$ receive shares of 1 from the i th instance of batch ssPMT. Otherwise, $y_i \notin X_j$, $\mathcal{S}$ and $\mathcal{R}$ receive shares of 0.

The functionality realized by the above construction differs slightly from mq-ssPMT, as the query sequence of $\mathcal{R}$ is determined by the Cuckoo hash positions of its input elements. Since the Cuckoo hash positions depends on the entire input set of $\mathcal{R}$, and all shares is arranged according to the parties' Cuckoo hash positions, a straightforward reconstruction may leak information about the parties' input sets to P_1 . Therefore, we have to eliminate the dependence of shares' order on the parties' Cuckoo hash positions during the reconstruction phase. Our crucial insight is that the multi-party secret-shared shuffle used in the reconstruction phase eliminates not only the correspondence between secret shares and difference sets, but also this dependence on the hash positions. As a result, our construction can plug in the secret-sharing based framework seamlessly, without introducing any information leakage or additional overhead.

Unlike the mq-ssPMT, which heavily relies on general 2PC, our batch ssPMT is built on two highly efficient primitives, batched oblivious programmable pseudorandom function (batch OPRF) [39, 49] and secret-shared private equality

test (ssPEQT) [49, 16], where the former has an extremely fast specialized instantiation through vector oblivious linear evaluation (VOLE), while the later is composed of a small circuit, allowing for efficient implementation with general 2PC. Therefore, our batch ssPMT greatly reduces dependency on general 2PC, leading to a substantial performance improvement (refer to Section 7.1).

2.3 SK-MPSU from Batch ssPMT and mss-ROT

We present our SK-MPSU in an incremental way over the secret-sharing based framework. Our first attempt to construct a secret-sharing process for each P_j ($2 \leq j \leq m$) to secret-share Y_j is as follows: Let P_j preprocess X_j via Cuckoo hashing while each P_i ($1 \leq i < j$) preprocesses X_i via simple hashing. P_j acts as $\mathcal{R}$ and executes the batch ssPMT with each P_i . For each bin, P_j and P_i receive shares $e_{j,i}$ and $e_{i,j}$, respectively. Then P_j acts as $\mathcal{S}$ and executes the two-choice-bit ROT with each P_i , where P_j inputs $e_{j,i}$ and P_i inputs $e_{i,j}$. P_j receives $r_{j,i}^0, r_{j,i}^1$. If $e_{j,i} \oplus e_{i,j} = 0$, P_i receives $r_{j,i}^0$, otherwise, P_i receives $r_{j,i}^1$. P_i sets the output $r_{j,i}^{e_{j,i} \oplus e_{i,j}}$ as its share $s_{j,i}$. P_j sets its share $s_{j,j}$ to be $\bigoplus_{i=1}^{j-1} r_{j,i}^0 \oplus x \| H(x)$, where $x \in X_j$ is the element in P_j 's bin and the appended hash value $H(x)$ is to distinguish x and a random secret in the reconstruction. If $x \in Y_j$, then $e_{j,i} \oplus e_{i,j} = 0$ holds for all i , so we have $\bigoplus_{i=1}^j s_{j,i} = x \| H(x)$, i.e., the parties hold secret shares of $x \| H(x)$. Otherwise, at least one $r_{j,i}^0 \oplus r_{j,i}^1$ cannot be canceled out in the summation, so the parties hold a random secret-sharing.

Integrating the above construction into the secret-sharing based framework results in a more efficient MPSU protocol compared to LG. However, this protocol remains vulnerable to the same colluding attacks. For instance, consider the case of P_1 reconstructing a secret as $r_{j,i'}^0 \oplus r_{j,i'}^1 \oplus x \| H(x)$ for some $1 \leq i' < j$ in the second phase. Since P_j receives $r_{j,i'}^0, r_{j,i'}^1$ from the ROT execution with $P_{i'}$, colluding with P_1 allows P_1 to infer that $e_{j,i} \oplus e_{i,j} = 0$ holds for all $i \neq i'$. Then they can derive that $x \in X_{i'}$ and $x \notin X_i$ for all $i \neq i'$, which reveals information about $P_2, \dots, P_{j-1}$. Therefore, both the resulting protocol and LG requires the non-collusion assumption that P_1 does not collude with other parties.

We identified that this vulnerability stems from the secrets of random secret-sharings that are not uniformly random in the view of P_1 and other colluding parties. Furthermore, the re-sharing of multi-party secret-shared shuffle in the second phase retains these secrets. Therefore, to resist the colluding attacks, our solution is to additively secret-share each $\Delta_{ji} = r_{j,i}^0 \oplus r_{j,i}^1$ among $\{P_2, \dots, P_j\}$, so that each potential remaining component of the summation is uniformly random, as long as there is at least one honest party in $\{P_2, \dots, P_j\}$. To realize this additive secret-sharing, we generalize the two-choice-bit ROT into multi-party setting, abstracting the notion of multi-party secret-shared random oblivious transfer (mss-ROT). Consider j parties engaging in the mss-ROT, where there are two parties, denoted as P_{ch_0} and P_{ch_1} , possessing the choice bits b_0 and b_1 respectively. There is a subset J of all involved parties s.t. each $P_{j'} \in J$ holds $\Delta_{j'}$ as input. Each involved party $P_{i'}$ a random output $r_{i'}$ s.t. if $b_0 \oplus b_1 = 0$, $\bigoplus_{i'=1}^j r_{i'} = 0$. Otherwise, $\bigoplus_{i'=1}^j r_{i'} = \bigoplus_{j' \in J} \Delta_{j'}$.

Equipped with mss-ROT, the secret-sharing process of P_j is revised as follows: For each bin, after P_j and P_i receiving shares $e_{j,i}$ and $e_{i,j}$, they invoke mss-ROT instead of two-choice-bit ROT, with $J = \{P_2, \dots, P_j\}$ and each $P_{j'} \in J$ inputting a uniformly sampled $\Delta_{j',ji}$. Each involved party in mss-ROT receives a random output. By the mss-ROT functionality, if $e_{j,i} \oplus e_{i,j} = 0$, all outputs XOR to 0, otherwise, they XOR to $\bigoplus_{j'=2}^j \Delta_{j',ji}$. At the end of the process, each party XORs all received outputs from different invocations of mss-ROT as its share. P_j additionally XORs $x \parallel H(x)$. If $x \in Y_j$, then $e_{j,i} \oplus e_{i,j} = 0$ holds for all i , hence all the outputs from mss-ROT XOR to 0, and the shares XOR to $x \parallel H(x)$. Otherwise, at least one random $\bigoplus_{j'=2}^j \Delta_{j',ji}$ remains, which is additively secret-shared among $\{P_2, \dots, P_j\}$. Hence, the parties hold a secret-sharing with a random secret, even in the view of P_1 colluding with up to $m-2$ parties in $\{P_2, \dots, P_j\}$. Given that any coalition without P_1 cannot know the reconstructed secrets, the protocol is secure against arbitrary collusion of $m-1$ parties.

2.4 PK-MPSU from Batch ssPMT and MKR-PKE

There are no existing MPSU protocols with linear computation and communication complexity. Notably, it is impossible to achieve this goal within the secret-sharing based framework, given that P_1 must reconstruct $(m-1)n$ secrets with each secret comprising m shares. Therefore, we have to seek for another technical route to construct MPSU.

Our start point is the existing work [28] with the best asymptotic complexity $mn(\log n / \log \log n)$. We distill their protocol into a encryption-based framework (using a PKE variant MKR-PKE) with two phases: The first phase allows P_1 to somehow obtain the encrypted $Y_j = X_j \setminus (X_1 \cup \dots \cup X_{j-1})$ from each P_j ($2 \leq j \leq m$), interspersed with encrypted dummy messages filling the positions of duplicate elements. If P_1 decrypts these ciphertexts by itself, then it could associate each reconstructed element with the specific party it originates from. The second phase addresses this issue through a collaborative decryption and shuffle procedure, which enables P_1 to obtain all plaintexts in a random permutation, and prevents any coalition of $m-1$ parties from knowing the permutation.

We identify that both the non-linear complexities and insecurity against arbitrary collusion of [28] arise from their construction in the first phase. Therefore, the task of building a linear-complexity MPSU reduces to devising a linear-complexity and secure construction for this phase.

We utilize the batch mq-ssPMT and two-choice-bit OT to achieve this task. A rough idea is as follows: For $2 \leq j \leq m$, each P_j first encrypts the elements in X_j and then engages in a procedure sequentially with each P_i ($1 < i < j$), that allows P_i to “obliviously” replace the encrypted elements in $X_j \cap X_i$ with encrypted dummy messages. Specifically, after preprocessing their inputs using hashing-to-bins as previous, P_j acts as $\mathcal{R}$ and executes batch ssPMT with P_i . For each bin, P_j and P_i receive shares $e_{j,i}$ and $e_{i,j}$. If $i = 2$, P_j initializes m_0 as the encrypted x (where $x \in X_j$ is the element in P_j ’s bin) and m_1 as an encrypted dummy message. P_j acts as $\mathcal{S}$ and executes two-choice-bit OT with

P_i , where P_j and P_i input $e_{j,i}$ and $e_{i,j}$ as choice bits. P_j inputs m_0, m_1 and P_i receives $\text{ct} = m_{e_{j,i} \oplus e_{i,j}}$. If $x \notin X_i$, ct is the ciphertext of x ; otherwise, ct is the ciphertext of dummy message. P_i rerandomizes ct to ct' and returns ct' to P_j . Then P_j rerandomizes ct' , updates m_0 to ct' , and rerandomizes m_1 before repeating the procedure with the next party P_{i+1} . After interacting with all P_i ($1 < i < j$), P_j retains an encrypted x in the final m_0 , only if $x \notin X_i$ holds for all i . As a result, P_j holds the encrypted elements in $X_j \setminus (X_2 \cup \dots \cup X_{j-1})$. Finally, replacing the encrypted elements in X_1 with dummy messages is handled in a similar manner at the end of this procedure.

The linear complexities of our construction are attributed to two key facts: First, this framework ensures that each interaction between P_j and P_i only involves the input sets of themselves. Second, our batch ssPMT protocol has linear computation and communication complexity (cf. Section 4).

3 Preliminaries

3.1 Notation

Let m denote the number of participants. We write $[m]$ to denote $\{1, \dots, m\}$. We use P_i ($i \in [m]$) to denote participants, X_i to represent the sets they hold, where each set has n l -bit elements. $x \parallel y$ denotes the concatenation of two strings. We use λ, σ as the computational and statistical security parameters respectively, and use $\overset{s}{\approx}$ (resp. $\overset{c}{\approx}$) to denote that two distributions are statistically (resp. computationally) indistinguishable. We denote vectors by letters with a right arrow above and a_i denotes the i th component of $\vec{a}$. $\vec{a} \oplus \vec{b} = (a_1 \oplus b_1, \dots, a_n \oplus b_n)$. $\pi(\vec{a}) = (a_{\pi(1)}, \dots, a_{\pi(n)})$, where π is a permutation over n items. $\pi = \pi_1 \circ \dots \circ \pi_n$ represents applying the permutations $\pi_1, \dots, \pi_n$ in sequence. $x[i]$ denotes the i th bit of element x , and $X(i)$ denotes the i th element of set X .

3.2 Security Model

In this work, we consider a semi-honest and static adversary $\mathcal{A}$ with the capability to corrupt an arbitrary subset of $t < m$ parties (i.e. the corruption threshold $t = m - 1$). To define the standard semi-honest security as in [31, 14], let $f = (f_1, \dots, f_m)$ be a probabilistic polynomial-time m -ary functionality and let Π be a m -party protocol for computing f . The view of P_i ($1 \leq i \leq m$) during an execution of Π on all parties' inputs $\mathbf{x} = (x_1, \dots, x_m)$ is denoted by $\text{View}_i^\Pi(\mathbf{x})$. The output of P_i during an execution of Π on $\mathbf{x}$ is denoted by $\text{Output}_i^\Pi(\mathbf{x})$. The joint output of parties is $\text{Output}^\Pi(\mathbf{x}) = (\text{Output}_1^\Pi(\mathbf{x}), \dots, \text{Output}_m^\Pi(\mathbf{x}))$.

Definition 1. We say that Π securely computes f in the presence of $\mathcal{A}$ if there exists a PPT algorithm Sim s.t. for every $\mathbf{P}_\mathcal{A} = \{P_{i_1}, \dots, P_{i_t}\} \subset \{P_1, \dots, P_m\}$,

$$\{\text{Sim}(\mathbf{P}_\mathcal{A}, \mathbf{x}_\mathcal{A}, f_\mathcal{A}(\mathbf{x})), f(\mathbf{x})\}_x \overset{c}{\approx} \{\text{View}_\mathcal{A}^\Pi(\mathbf{x}), \text{Output}^\Pi(\mathbf{x})\}_x,$$

where $\mathbf{x}_\mathcal{A} = (x_{i_1}, \dots, x_{i_t})$, $f_\mathcal{A} = (f_{i_1}, \dots, f_{i_t})$, $\text{View}_\mathcal{A}^\Pi(\mathbf{x}) = (\text{View}_{i_1}^\Pi(\mathbf{x}), \dots, \text{View}_{i_t}^\Pi(\mathbf{x}))$.

3.3 Multi-Party Private Set Union

The ideal functionality of MPSU is formalized in Figure 2.

Parameters. m parties $P_1, \dots, P_m$, where P_1 is the leader. Size n of input sets. The bit length l of set elements.

Functionality. On input $X_i = \{x_i^1, \dots, x_i^n\} \subseteq \{0, 1\}^l$ from P_i ($i \in [m]$), output $\bigcup_{i=1}^m X_i$ to P_1 .

Fig. 2. Multi-party Private Set Union Functionality $\mathcal{F}_{\text{mpsu}}$

3.4 Batch Oblivious Programmable Pseudorandom Function

The (batch) oblivious programmable pseudorandom function (OPPRF) [39, 49, 16, 55, 52] is an extension of (batch) oblivious pseudorandom function (OPRF) [26, 38]. The batch OPPRF functionality is given in Figure 3.

Parameters. Sender $\mathcal{S}$. Receiver $\mathcal{R}$. Batch size B . The bit length l of keys. The bit length γ of values.

Sender's inputs. $\mathcal{S}$ inputs B sets of key-value pairs including:

- Disjoint key sets $K_1, \dots, K_B$.
- The value sets $V_1, \dots, V_B$, where $|K_i| = |V_i|$, $i \in [B]$.

Receiver's inputs. $\mathcal{R}$ inputs B queries $\vec{x} \subseteq (\{0, 1\}^l)^B$.

Functionality: On input $(K_1, \dots, K_B)$ and $(V_1, \dots, V_B)$ from $\mathcal{S}$ and $\vec{x} \subseteq (\{0, 1\}^l)^B$ from $\mathcal{R}$,

- Choose a uniform PPRF key k_i , for $i \in [B]$;
- Sample a PPRF $F : \{0, 1\}^* \times \{0, 1\}^l \rightarrow \{0, 1\}^\gamma$ such that $F(k_i, K_i(j)) = V_i(j)$ for $i \in [B], 1 \leq j \leq |K_i|$;
- Define $f_i = F(k_i, x_i)$, for $i \in [B]$;
- Give vector $\vec{f} = (f_1, \dots, f_B)$ to $\mathcal{R}$.

Fig. 3. Batch OPPRF Functionality $\mathcal{F}_{\text{bOPPRF}}$

3.5 Hashing to Bins

The hashing-to-bins technique [50, 48] is used in PSO to ensure that identical items across parties are assigned into bins with identical indices. This align-

ment is achieved by preprocessing input sets via simple hashing and Cuckoo hashing [45].

We denote simple hashing with the following notation:

$$\mathcal{T}^1, \dots, \mathcal{T}^B \leftarrow \text{Simple}_{h_1, h_2, h_3}^B(X)$$

This expression represents hashing items in X into B bins using simple hashing with hash functions $h_1, h_2, h_3 : \{0, 1\}^* \rightarrow [B]$. The output is a simple hash table denoted by $\mathcal{T}^1, \dots, \mathcal{T}^B$, where for each $x \in X$, $\mathcal{T}^{h_i(x)} \supseteq \{x \parallel i \mid i = 1, 2, 3\}$.⁶

We denote Cuckoo hashing with the following notation:

$$\mathcal{C}^1, \dots, \mathcal{C}^B \leftarrow \text{Cuckoo}_{h_1, h_2, h_3}^B(X)$$

This expression represents hashing items in X into B bins using Cuckoo hashing with hash functions h_1, h_2, h_3 . The output is a Cuckoo hash table denoted by $\mathcal{C}^1, \dots, \mathcal{C}^B$, where for each $x \in X$ there is some $i \in \{1, 2, 3\}$ s.t. $\mathcal{C}^{h_i(x)} = \{x \parallel i\}$.

3.6 Secret-Shared Private Equality Test

The secret-shared private equality test protocol (ssPEQT) [49, 16] can be viewed as an extreme case of ssPMT when the sender $\mathcal{S}$'s input set size is 1. Figure 4 defines its functionality.

Parameters. Two parties P_1, P_2 . The input bit length γ .
Functionality. On input x from P_1 and input y from P_2 , sample two random bits a, b s.t. if $x = y$, $a \oplus b = 1$. Otherwise $a \oplus b = 0$. Give a to P_1 and b to P_2 .

Fig. 4. Secret-Shared Private Equality Test Functionality $\mathcal{F}_{\text{ssPEQT}}$

3.7 Random Oblivious Transfer

Oblivious transfer (OT) [51] is a foundational primitive in MPC, the functionality of 1-out-of-2 random OT (ROT) is given in Figure 5.

3.8 Multi-Party Secret-Shared Shuffle

Multi-party secret-shared shuffle functionality works by randomly permuting the share vectors of all parties and then refreshing all shares, ensuring that the permutation remains unknown to any coalition of $m - 1$ parties. The formal functionality is given in Figure 6.

⁶ Appending the index of the hash function is helpful for dealing with edge cases like $h_1(x) = h_2(x) = i$, which happen with non-negligible probability.

Parameters. Sender $\mathcal{S}$, Receiver $\mathcal{R}$. The message length l .
Functionality. On input $b \in \{0, 1\}$ from $\mathcal{R}$, sample $r_0, r_1 \leftarrow \{0, 1\}^l$. Give (r_0, r_1) to $\mathcal{S}$ and give r_b to $\mathcal{R}$.

Fig. 5. 1-out-of-2 Random OT Functionality $\mathcal{F}_{\text{rot}}$

Parameters. m parties $P_1, \dots, P_m$. The dimension of vector n . The item length l .
Functionality. On input $\vec{x}_i = (x_i^1, \dots, x_i^n)$ from each P_i , sample a random permutation $\pi : [n] \rightarrow [n]$. For $1 \leq i \leq m$, sample $\vec{x}_i' \leftarrow (\{0, 1\}^l)^n$ satisfying $\bigoplus_{i=1}^m \vec{x}_i' = \pi(\bigoplus_{i=1}^m \vec{x}_i)$. Give $\vec{x}_i'$ to P_i .

Fig. 6. Multi-Party Secret-Shared Shuffle Functionality $\mathcal{F}_{\text{ms}}$

3.9 Multi-Key Rerandomizable Public-Key Encryption

Gao et al. [28] introduced the concept of multi-key rerandomizable public-key encryption (MKR-PKE), a variant of PKE with several additional properties. Let $\mathcal{SK}$ denote the space of secret keys, which forms an abelian group under the operation $+$, and $\mathcal{PK}$ denote the space of public keys, which forms an abelian group under the operation $\cdot$. $\mathcal{M}$ denotes the space of plaintexts, and $\mathcal{C}$ denotes the space of ciphertexts. MKR-PKE is a tuple of PPT algorithms $(\text{Gen}, \text{Enc}, \text{ParDec}, \text{Dec}, \text{ReRand})$ such that:

- The key-generation algorithm Gen takes as input a security parameter 1^λ and outputs a pair of keys $(pk, sk) \in \mathcal{PK} \times \mathcal{SK}$.
- The encryption algorithm Enc takes as input a public key $pk \in \mathcal{PK}$ and a plaintext message $x \in \mathcal{M}$, and outputs a ciphertext $\text{ct} \in \mathcal{C}$.
- The partial decryption algorithm ParDec takes as input a secret key share $sk \in \mathcal{SK}$ and a ciphertext $\text{ct} \in \mathcal{C}$, and outputs another ciphertext $\text{ct}' \in \mathcal{C}$.
- The decryption algorithm Dec takes as input a secret key $sk \in \mathcal{SK}$ and a ciphertext $\text{ct} \in \mathcal{C}$, outputs a message $x \in \mathcal{M}$ or an error symbol $\perp$.
- The rerandomization algorithm ReRand takes as input a public key $pk \in \mathcal{PK}$ and a ciphertext $\text{ct} \in \mathcal{C}$, outputs another ciphertext $\text{ct}' \in \mathcal{C}$.

MKR-PKE is an IND-CPA secure PKE scheme with additional properties:

Partially Decryptable. For any two pairs of keys $(sk_1, pk_1) \leftarrow \text{Gen}(1^\lambda), (sk_2, pk_2) \leftarrow \text{Gen}(1^\lambda)$ and any $x \in \mathcal{M}$,

$$\text{ParDec}(sk_1, \text{Enc}(pk_1 \cdot pk_2, x)) \stackrel{s}{\approx} \text{Enc}(pk_2, x)$$

Rerandomizable. For any $pk \in \mathcal{PK}$ and any $x \in \mathcal{M}$,

$$\text{ReRand}(pk, \text{Enc}(pk, x)) \stackrel{s}{\approx} \text{Enc}(pk, x)$$

Gao et al. [28] make use of elliptic curve (EC) based ElGamal encryption [24] to instantiate MKR-PKE. The concrete EC MKR-PKE is given in Appendix B.2.

Remark 1. To the best of our knowledge, there is few elliptic curves that support efficient encoding and decoding between bit-strings and EC points. Therefore, the plaintext space of EC MKR-PKE is generally restricted to EC points to guarantee rerandomizable property.

4 Batch Secret-Shared Private Membership Test

The batch secret-shared private membership test (batch ssPMT) is a central building block in our SK-MPSU and PK-MPSU protocols. It is a two-party protocol that implements multiple instances of ssPMT between a sender $\mathcal{S}$ and a receiver $\mathcal{R}$. Given a batch size of B , $\mathcal{S}$ inputs B sets $X_1, \dots, X_B$, while $\mathcal{R}$ inputs B elements $x_1, \dots, x_B$. As a result, $\mathcal{S}$ and $\mathcal{R}$ receive secret shares of a bit vector of size B , where the i th bit is 1 if $x_i \in X_i$, 0 otherwise. The batch ssPMT functionality is presented in Figure 7 and the construction is given in Figure 8, built from batch OPPRF and ssPEQT.

Parameters. Sender $\mathcal{S}$. Receiver $\mathcal{R}$. Batch size B . The bit length l of set elements. The output length γ of OPPRF.

Inputs. $\mathcal{S}$ inputs B disjoint sets $X_1, \dots, X_B$ and $\mathcal{R}$ inputs $\vec{x} \subseteq (\{0, 1\}^l)^B$.

Functionality. On inputs $X_1, \dots, X_B$ from $\mathcal{S}$ and input $\vec{x}$ from $\mathcal{R}$, for each $i \in [B]$, sample two random bits e_S^i, e_R^i s.t. if $x_i \in X_i$, $e_S^i \oplus e_R^i = 1$, otherwise $e_S^i \oplus e_R^i = 0$. Give $\vec{e}_S = (e_S^1, \dots, e_S^B)$ to $\mathcal{S}$ and $\vec{e}_R = (e_R^1, \dots, e_R^B)$ to $\mathcal{R}$.

Fig. 7. Batch ssPMT Functionality $\mathcal{F}_{\text{bssPMT}}$

Theorem 1. *Protocol Π_{bssPMT} securely realizes $\mathcal{F}_{\text{bssPMT}}$ in the $(\mathcal{F}_{\text{bOPPRF}}, \mathcal{F}_{\text{ssPEQT}})$ -hybrid model.*

Correctness. By batch OPPRF functionality, if $x_i \in X_i$, $\mathcal{R}$ receives $t_i = s_i$. Then in the i th instance of ssPEQT, $s_i = t_i$, $e_S^i \oplus e_R^i = 1$. Conversely, if $x_i \notin X_i$, $\mathcal{R}$ receives a pseudorandom value t_i . The probability that any $t_i = s_i$ for $i \in [B]$ is $B \cdot 2^{-\gamma}$. By setting $\gamma \geq \sigma + \log B$, we have $B \cdot 2^{-\gamma} \leq 2^{-\sigma}$, meaning the probability of any such collision is negligible (In MPSU, the batch OPPRF is invoked multiple times, and the lower bound of γ is relevant to the total number of batch OPPRF invocations. Refer to Section 7.1 for details). After the invocation of ssPEQT, if $x_i \notin X_i$, $e_S^i \oplus e_R^i = 0$ with overwhelming probability.

Security. The security of the protocol follows immediately from the security of batch OPPRF and ssPEQT functionalities.

Parameters. Sender $\mathcal{S}$. Receiver $\mathcal{R}$. Batch size B . The bit length l of set elements. The output length γ of OPPRF.

Inputs. $\mathcal{S}$ inputs B disjoint sets $X_1, \dots, X_B$ and $\mathcal{R}$ inputs $\vec{x} \subseteq (\{0, 1\}^l)^B$.

Protocol.

1. For each $i \in [B]$, $\mathcal{S}$ chooses random $s_i \leftarrow \{0, 1\}^\gamma$ and computes a multiset S_i comprising $|X_i|$ repeated elements that all equal to s_i .
2. The parties invoke $\mathcal{F}_{\text{bOPPRF}}$ of batch size B . $\mathcal{S}$ acts as sender and inputs $X_1, \dots, X_B$ as key sets and $S_1, \dots, S_B$ as value sets. $\mathcal{R}$ acts as receiver with input $\vec{x}$, and receives a vector $\vec{t} = (t_1, \dots, t_B)$.
3. The parties invoke B instances of $\mathcal{F}_{\text{ssPEQT}}$, where in the i th instance $\mathcal{S}$ inputs s_i and $\mathcal{R}$ inputs t_i . In the end, $\mathcal{S}$ and $\mathcal{R}$ receives $e_S^i, e_R^i \in \{0, 1\}$ respectively.

Fig. 8. Batch ssPMT Π_{bssPMT}

In our MPSU protocols, the batch ssPMT is always combined with the hashing-to-bins technique. The combined construction has good efficiency, together with linear computation and communication in term of n , which mainly benefits from the following technical advances: First, we follow the paradigm in [49] to construct batch OPPRF from batch OPRF and oblivious key-value store (OKVS) [47, 30, 52, 7]. By leveraging the technique to amortize communication, the total communication of computing $B = O(n)$ instances of OPPRF is equal to the total number of items $3n$. Second, we utilize subfield vector oblivious linear evaluation (subfield-VOLE) [9, 10, 53] to instantiate batch OPRF and the construction in [52] to instantiate OKVS. This ensures the computation complexity of batch OPPRF of size $O(n)$ to scale linearly with n .

5 MPSU from Symmetric-Key Techniques

In this section, we introduce a new primitive called multi-party secret-shared random oblivious transfer (mss-ROT), and utilize it to build SK-MPSU based on oblivious transfer and symmetric-key operations in standard semi-honest model.

5.1 Multi-Party Secret-Shared Random Oblivious Transfer

In Figure 5, the ROT functionality can be interpreted as $r_0 \oplus r_b = b \cdot \Delta$, where $\Delta = r_0 \oplus r_1$. As the aforementioned two-choice-bit ROT, it can be consider to secret-share the choice bit b between two parties and interpreted as $r_0 \oplus r_b = (b_0 \oplus b_1) \cdot \Delta$. We further extend this additive secret-sharing idea to allow any non-empty subset of the involved parties to secret-share Δ . The mss-ROT functionality is formally given in Figure 9 with a construction in Figure 10.

As a matter of fact, the secret-sharing approach is commonly used in MPC. For instance, general MPC protocols such as SPDZ [23, 22] and MASCOT [36]

Parameters. m parties $P_1, \dots, P_m$, where P_{ch_0} and P_{ch_1} provide inputs as shares of the choice bit, $ch_0, ch_1 \in [m]$. We use $J = \{j_1, \dots, j_d\}$ to denote the set of indices for the parties who provide inputs as shares of Δ , where $d \leq m$. The message length l .

Functionality. On input $b_0 \in \{0, 1\}$ from P_{ch_0} , $b_1 \in \{0, 1\}$ from P_{ch_1} , and Δ_j from each P_j where $j \in J$,

- Sample $r_i \leftarrow \{0, 1\}^l$ and give r_i to P_i for $2 \leq i \leq m$.
- If $b_0 \oplus b_1 = 0$, compute $r_1 = \bigoplus_{i=2}^m r_i$, else compute $r_1 = \bigoplus_{i=2}^m r_i \oplus (\bigoplus_{j \in J} \Delta_j)$. Give r_1 to P_1 .

Fig. 9. Multi-Party Secret-Shared Random OT Functionality $\mathcal{F}_{\text{mss-rot}}$

Parameters. m parties $P_1, \dots, P_m$. where P_{ch_0} and P_{ch_1} provide inputs as shares of the choice bit, $ch_0, ch_1 \in [m]$. We use $J = \{j_1, \dots, j_d\}$ to denote the set of indices for the parties who provide inputs as shares of Δ , where $d \leq m$. The message length l .

Inputs. P_{ch_0} has input $b_0 \in \{0, 1\}$ and P_{ch_1} has input $b_1 \in \{0, 1\}$. Each P_j for $j \in J$ has input Δ_j ,

Protocol.

1. For $1 \leq i \leq m$, P_i initializes $r_i = 0$. If $ch_{j=0,1} \in J$, P_{ch_j} sets $r_{ch_j} = b_j \cdot \Delta_{ch_j}$ ($\cdot$ denotes bitwise-AND).
2. For $j \in J$: P_{ch_0} and P_j invoke $\mathcal{F}_{\text{rot}}$ where P_{ch_0} acts as receiver with input b_0 and P_j as sender. P_{ch_0} receives $u_{j,ch_0}^{b_0} \in \{0, 1\}^l$. P_j receives $u_{j,ch_0}^0, u_{j,ch_0}^1 \in \{0, 1\}^l$. P_j updates $r_j = r_j \oplus u_{j,ch_0}^0$ and sends $\Delta'_{j,ch_0} = u_{j,ch_0}^0 \oplus u_{j,ch_0}^1 \oplus \Delta_j$ to P_{ch_0} . P_{ch_0} updates $r_{ch_0} = r_{ch_0} \oplus u_{j,ch_0}^{b_0} \oplus b_0 \cdot \Delta'_{j,ch_0}$.
3. For $j \in J$: P_{ch_1} and P_j invoke $\mathcal{F}_{\text{rot}}$ where P_{ch_1} acts as receiver with input b_1 and P_j as sender. P_{ch_1} receives $u_{j,ch_1}^{b_1} \in \{0, 1\}^l$. P_j receives $u_{j,ch_1}^0, u_{j,ch_1}^1 \in \{0, 1\}^l$. P_j updates $r_j = r_j \oplus u_{j,ch_1}^0$ and sends $\Delta'_{j,ch_1} = u_{j,ch_1}^0 \oplus u_{j,ch_1}^1 \oplus \Delta_j$ to P_{ch_1} . P_{ch_1} updates $r_{ch_1} = r_{ch_1} \oplus u_{j,ch_1}^{b_1} \oplus b_1 \cdot \Delta'_{j,ch_1}$.
4. If $|I = \{ch_0, ch_1\} \cup J| < m$, each P_i ($i \in I$) samples $r'_{i,i'}$ and sends $r'_{i,i'}$ to each $P_{i'}$ ($i' \in [m] \setminus I$) and updates $r_i = r_i \oplus r'_{i,i'}$. $P_{i'}$ updates $r_{i'} = r_{i'} \oplus r'_{i,i'}$.
5. For $1 \leq i \leq m$, P_i outputs the final value of r_i .

Fig. 10. Multi-Party Secret-Shared Random OT $\Pi_{\text{mss-rot}}$

leverage secret-sharing to construct authenticated values, which bears structural similarities to the mss-ROT functionality. However, since mss-ROT is tailored to address the leakage caused by collusion between P_1 and other parties in the secret-sharing MPSU framework, it has the following distinct requirements: (1) The additive secret shares of Δ is distributed among the parties that could potentially collude with P_1 and cause leakage in each invocation. Since these colluding parties form a subset of the involved parties, the allocation of additive secret shares of Δ is defined by a subset J . (2) The choice bit in mss-ROT is secret-shared between only two parties to be plugged into the secret-sharing framework. (3) Since all shares are reconstructed in the final phase, the Δ in mss-ROT cannot be reused across multiple invocations.

Theorem 2. *Protocol $\Pi_{\text{mss-rot}}$ securely implements $\mathcal{F}_{\text{mss-rot}}$ in the presence of any semi-honest adversary corrupting $t < m$ parties in the $\mathcal{F}_{\text{rot}}$ -hybrid model.*

We consider the case that $\text{ch}_0 = 1$ and $\text{ch}_1 = m$, and $J = \{1, \dots, m\}$. Let P_1 and P_m input $b_1 \in \{0, 1\}$ and $b_m \in \{0, 1\}$, and P_i ($1 \leq i \leq m$) input Δ_i respectively. We turn to proving the correctness and security of the protocol in Figure 10 in this particular case. Note that in the different cases, the proof is essentially the same.

Correctness. From the description of the protocol, we have the following equations:

$$r_1 = \bigoplus_{j=2}^m (r_{j,1}^{b_1} \oplus b_1 \cdot u_{j,1}) \oplus r_{1,m}^0 \oplus b_1 \cdot \Delta_1, \quad (1)$$

$$r_i = r_{i,1}^0 \oplus r_{i,m}^0, 1 < i < m, \quad (2)$$

$$r_m = \bigoplus_{j=1}^{m-1} (r_{j,m}^{b_m} \oplus b_m \cdot u_{j,m}) \oplus r_{m,1}^0 \oplus b_m \cdot \Delta_m, \quad (3)$$

$$u_{j,1} = \Delta_j \oplus r_{j,1}^0 \oplus r_{j,1}^1, u_{j,m} = \Delta_j \oplus r_{j,m}^0 \oplus r_{j,m}^1 \quad (4)$$

From the definition of Random OT functionality (Figure 5), we have the following equations:

$$r_{j,1}^{b_1} = r_{j,1}^0 \oplus b_1 \cdot (r_{j,1}^0 \oplus r_{j,1}^1), \quad (5)$$

$$r_{j,m}^{b_m} = r_{j,m}^0 \oplus b_m \cdot (r_{j,m}^0 \oplus r_{j,m}^1), \quad (6)$$

Substitute Equation 4, 5, 6 into Equation 1, 2, 3 and cancel out the same terms, we obtain:

$$r_1 = \bigoplus_{j=2}^m (r_{j,1}^{b_1} \oplus b_1 \cdot \Delta_j) \oplus r_{1,m}^0 \oplus b_1 \cdot \Delta_1, \quad (7)$$

$$r_m = \bigoplus_{j=1}^{m-1} (r_{j,m}^{b_m} \oplus b_m \cdot \Delta_j) \oplus r_{m,1}^0 \oplus b_m \cdot \Delta_m, \quad (8)$$

Substitute Equation 2, 7, 8 into $r_1 \oplus (\bigoplus_{j=2}^{m-1} r_j) \oplus r_m$, we have $r_1 \oplus (\bigoplus_{j=2}^{m-1} r_j) \oplus r_m = \bigoplus_{i=1}^m (b_1 \oplus b_m) \cdot \Delta_i$. Then we can summarize that if $b_1 \oplus b_m = 0$, $r_1 = \bigoplus_{j=2}^m r_j$, else $r_1 = \Delta_1 \oplus (\bigoplus_{j=2}^m (r_j \oplus \Delta_j))$. This is exactly $\mathcal{F}_{\text{mss-rot}}$.

Security. We now prove the security of the protocol.

Proof. Let Corr denote the set of all corrupted parties and $\mathcal{H}$ denote the set of all honest parties. $|\text{Corr}| = t$.

Intuitively, the protocol is secure because all things the parties do are invoking $\mathcal{F}_{\text{rot}}$ and receiving random messages. The simulator can easily simulate these outputs from $\mathcal{F}_{\text{rot}}$ and protocol messages by generating random values, which are independent of honest parties' inputs.

To elaborate, in the case that $P_1 \notin \text{Corr}$ and $P_m \notin \text{Corr}$, simulator receives all outputs r_c of $P_c \in \text{Corr}$ and needs to emulate each P_c 's view, including its private input Δ_c , outputs $(r_{c,1}^0, r_{c,1}^1)$ and $(r_{c,m}^0, r_{c,m}^1)$ from $\mathcal{F}_{\text{rot}}$. The simulator for corrupted P_c runs the protocol honestly except that it simulates uniform outputs from $\mathcal{F}_{\text{rot}}$ under the constraint that $r_{c,1}^0 \oplus r_{c,m}^0 = r_c$. Clearly, the joint distribution of all outputs r_c of $P_c \in \text{Corr}$, along with their view emulated by simulator, is indistinguishable from that in the real process.

In the case that $P_1 \in \text{Corr}$ or $P_m \in \text{Corr}$, since the protocol is symmetric with respect to the roles of P_1 and P_m , we focus on the case of corrupted P_1 . The simulator receives all outputs r_c of $P_c \in \text{Corr}$. For P_1 , its view consists of the choice bit b_1 , its private input Δ_1 , outputs $(r_{1,m}^0, r_{1,m}^1)$, $\{r_{j,1}^{b_1}\}_{1 < j \leq m}$ from $\mathcal{F}_{\text{rot}}$ and protocol messages $\{u_{j,1}\}_{1 < j \leq m}$ from P_j . For each $P_c (c \neq 1)$, its view consists of its private input Δ_c , outputs $(r_{c,1}^0, r_{c,1}^1)$ and $(r_{c,m}^0, r_{c,m}^1)$ from $\mathcal{F}_{\text{rot}}$.

For P_1 's view, the simulator runs the protocol honestly except that it simulates uniform outputs $(r_{1,m}^0, r_{1,m}^1)$, $r_{i,1}^{b_1}$ from $\mathcal{F}_{\text{rot}}$ and uniformly random messages $u_{i,1}$ from P_i under the constraint $\bigoplus_{j=2}^m (r_{j,1}^{b_1} \oplus b_1 \cdot u_{j,1}) \oplus r_{1,m}^0 \oplus b_1 \cdot \Delta_1 = r_1$, where $P_i \in \mathcal{H}$. For other corrupted parties' view, it runs the protocol honestly except that it sets $r_{c,m}^0 = r_{c,1}^0 \oplus r_c$ and simulates uniform output $r_{c,m}^1$ from $\mathcal{F}_{\text{rot}}$.

In the real execution, P_1 receives $u_{i,1} = \Delta_i \oplus r_{i,1}^0 \oplus r_{i,1}^1$. From the definition of ROT functionality, $r_{i,1}^0$ (when $b_1 = 0$) or $r_{i,1}^1$ (when $b_1 = 1$) is uniform and independent of P_1 's view. Therefore, $u_{i,1}$ is uniformly at random from the perspective of P_1 . Clearly, the joint distribution of all outputs r_c of $P_c \in \text{Corr}$, along with their view emulated by simulator, is indistinguishable from that in the real process.

In the case that $P_1 \in \text{Corr}$ and $P_m \in \text{Corr}$, the simulator receives all outputs r_c of $P_c \in \text{Corr}$. For P_1 , its view consists of the choice bit b_1 , its private input Δ_1 , outputs $(r_{1,m}^0, r_{1,m}^1)$, $\{r_{j,1}^{b_1}\}_{1 < j \leq m}$ from $\mathcal{F}_{\text{rot}}$ and protocol messages $\{u_{j,1}\}_{1 < j \leq m}$ from P_j . For each $P_c (c \neq 1, c \neq m)$, its view consists of its private input Δ_c , outputs $(r_{c,1}^0, r_{c,1}^1)$ and $(r_{c,m}^0, r_{c,m}^1)$ from $\mathcal{F}_{\text{rot}}$. For P_m , its view consists of its private input Δ_m , outputs $(r_{m,1}^0, r_{m,1}^1)$, $\{r_{j,m}^{b_m}\}_{1 \leq j < m}$ from $\mathcal{F}_{\text{rot}}$ and protocol messages $\{u_{j,m}\}_{1 \leq j < m}$ from P_j .

For P_1 's view, the simulator runs the protocol honestly except that it simulates uniform outputs $r_{i,1}^{b_1}$ from $\mathcal{F}_{\text{rot}}$ and uniformly random messages $u_{i,1}$ from P_i under the constraint $\bigoplus_{j=2}^m (r_{j,1}^{b_1} \oplus b_1 \cdot u_{j,1}) \oplus r_{1,m}^0 \oplus b_1 \cdot \Delta_1 = r_1$, where $P_i \in \mathcal{H}$.

For the view of $P_c (c \neq 1, c \neq m)$, it runs the protocol honestly except that it sets $r_{c,m}^0 = r_{c,1}^0 \oplus r_c$ and simulates uniform output $r_{c,m}^1$ from $\mathcal{F}_{\text{rot}}$. For P_m 's view, it runs the protocol honestly with the following changes:

- It simulates uniform outputs $r_{i,m}^{b_m}$ from $\mathcal{F}_{\text{rot}}$ and uniform messages $u_{i,m}$ from P_i under the constraint $\bigoplus_{j=1}^{m-1} (r_{j,m}^{b_m} \oplus b_m \cdot u_{j,m}) \oplus r_{m,1}^0 \oplus b_m \cdot \Delta_m = r_m$, where $P_i \in \mathcal{H}$.
- It sets the output $r_{c,m}^{b_m}$ from $\mathcal{F}_{\text{rot}}$ to be consistent with the partial view $(r_{c,m}^0, r_{c,m}^1)$ of each corrupted P_c in preceding simulation, where $c \neq 1$ and $c \neq m$.

Clearly, the joint distribution of all outputs r_c of $P_c \in \text{Corr}$, along with their view emulated by simulator, is indistinguishable from that in the real process.

5.2 Construction of Our SK-MPSU

We now turn our attention to construct SK-MPSU from batch ssPMT and mss-ROT. The construction follows the secret-sharing based framework outlined in Section 2.1 and 2.3, and it is formally presented in Figure 11.

In the first phase, each P_j ($2 \leq j \leq m$) perform the secret-sharing process: P_j assigns X_j into B bins via Cuckoo hashing. Each P_i ($1 \leq i < j$) assigns X_i into B bins via simple hashing. P_j acts as $\mathcal{R}$ and executes the batch ssPMT with each P_i . For each bin, P_j and P_i receive shares $e_{j,i}$ and $e_{i,j}$. Then $\{P_{\min(2,i)}, \dots, P_j\}$ invoke mss-ROT, where P_j and P_i act as P_{ch_0} and P_{ch_1} holding $e_{j,i}$ and $e_{i,j}$ respectively, while each $P_{j'} \in \{P_2, \dots, P_j\}$ samples $\Delta_{j',ji}$ uniformly as input. Each $P_{i'} \in \{P_{\min(2,i)}, \dots, P_j\}$ receives $r_{i',ji}$. If $e_{j,i} \oplus e_{i,j} = 0$, $\bigoplus_{i'=\min(2,i)}^j r_{i',ji} = 0$, otherwise $\bigoplus_{i'=\min(2,i)}^j r_{i',ji} = \bigoplus_{j'=2}^j \Delta_{j',ji}$. At the end of this process, P_1 sets $s_{1,j} = r_{1,j1}$, and each $P_{j'}$ sets $s_{j',j} = \bigoplus_{i=1}^{j-1} r_{j',ji}$, except for P_j , who sets $s_{j,j} = \bigoplus_{i=1}^{j-1} r_{j,ji} \oplus x \| \mathbf{H}(x)$, where $x \in X_j$ is the element in P_j 's bin. For those uninvolved parties, they set their corresponding shares to 0. We have $\bigoplus_{i=1}^j s_{i,j} = (\bigoplus_{i=1}^{j-1} (\bigoplus_{i'=\min(2,i)}^j r_{i',ji})) \oplus x \| \mathbf{H}(x)$. If $x \in X_j \setminus (X_1 \cup \dots \cup X_{j-1})$, then $e_{j,i} \oplus e_{i,j} = 0$ and $\bigoplus_{i'=\min(2,i)}^j r_{i',ji} = 0$ hold for all i , hence $\bigoplus_{i=1}^j s_{i,j} = x \| \mathbf{H}(x)$. This enables the parties to hold a secret-sharing of each element in $Y_j = X_j \setminus (X_1 \cup \dots \cup X_{j-1})$. Otherwise, at least one random $\bigoplus_{i'=\min(2,i)}^j r_{i',ji}$ cannot be canceled out and is additively secret-shared among $\{P_2, \dots, P_j\}$. Therefore, for each element in $X_j \setminus Y_j$, the parties hold a random secret-sharing whose secret reveals no information to P_1 even if it colludes with up to $m-2$ parties in $\{P_2, \dots, P_j\}$.

In the second phase, the parties invoke multi-party secret-shared shuffle to randomly permute and re-share all secret-sharings generated in the first phase. Afterwards, each P_j sends their shuffled shares to P_1 , who reconstructs $(X_1 \cup \dots \cup X_m) \setminus X_1$, appends the elements in X_1 , and obtains the union.

Theorem 3. *Protocol $\Pi_{\text{SK-MPSU}}$ securely implements $\mathcal{F}_{\text{mps}}$ against any semi-honest adversary corrupting $t < m$ parties in the $(\mathcal{F}_{\text{bssPMT}}, \mathcal{F}_{\text{mss-rot}}, \mathcal{F}_{\text{ms}})$ -hybrid model.*

Parameters. m parties $P_1, \dots, P_m$. Size n of input sets. The bit length l of set elements. Cuckoo hashing parameters: hash functions h_1, h_2, h_3 and number of bins B . A collision-resistant hash function $H(x) : \{0, 1\}^l \rightarrow \{0, 1\}^\kappa$. $\text{Elem}(\mathcal{C}^b)$ denotes the element in $\mathcal{C}^b$.

Inputs. Each party P_i inputs $X_i = \{x_i^1, \dots, x_i^n\} \subseteq \{0, 1\}^l$.

Protocol.

1. P_1 does $\mathcal{T}_1^1, \dots, \mathcal{T}_1^B \leftarrow \text{Simple}_{h_1, h_2, h_3}^B(X_1)$. For $1 < j \leq m$, P_j does $\mathcal{C}_j^1, \dots, \mathcal{C}_j^B \leftarrow \text{Cuckoo}_{h_1, h_2, h_3}^B(X_j)$ and $\mathcal{T}_j^1, \dots, \mathcal{T}_j^B \leftarrow \text{Simple}_{h_1, h_2, h_3}^B(X_j)$.
2. For $1 < j \leq m$:
 - For $1 \leq i < j$: P_i and P_j invoke $\mathcal{F}_{\text{bssPMT}}$ of batch size B , where P_i acts as sender with inputs $\mathcal{T}_i^1, \dots, \mathcal{T}_i^B$ and P_j acts as receiver with inputs $\mathcal{C}_j^1, \dots, \mathcal{C}_j^B$. For the instance $b \in [B]$, P_i receives $e_{i,j}^b \in \{0, 1\}$, and P_j receives $e_{j,i}^b \in \{0, 1\}$.
3. For $1 < j \leq m$:
 - For $1 \leq i < j$, for each bin $b \in [B]$: $P_{\min(2,i)}, \dots, P_j$ invoke $\mathcal{F}_{\text{mss-rot}}$ where P_j acts as P_{ch_0} with input $e_{j,i}^b$ and P_i acts as P_{ch_1} with input $e_{i,j}^b$. For $1 < j' \leq j$, $P_{j'}$ samples $\Delta_{j',ji}^b \leftarrow \{0, 1\}^{l+\kappa}$ and inputs $\Delta_{j',ji}^b$ as shares of Δ . For $\min(2,i) \leq i' \leq j$, $P_{i'}$ receives $r_{i',ji}^b \in \{0, 1\}^{l+\kappa}$.
 - P_1 sets $s_{j,1} = r_{1,j1}$. For $1 < j' < j$, $P_{j'}$ computes $s_{j',j}^b = \bigoplus_{i=1}^{j'-1} r_{j',ji}^b$. P_j computes $s_{j,j}^b = \bigoplus_{i=1}^{j-1} r_{j,ji}^b \oplus (\text{Elem}(\mathcal{C}_j^b) \parallel H(\text{Elem}(\mathcal{C}_j^b)))$ if $\mathcal{C}_j^b$ is not corresponding to an empty bin, otherwise chooses $s_{j,j}^b$ at random.
4. For $1 \leq i \leq m$, P_i computes $\vec{s}h_i \in (\{0, 1\}^{l+\kappa})^{(m-1)B}$ as follows: For $\max(2,i) \leq j \leq m, 1 \leq b \leq B$, $sh_{i,(j-2)B+b} = s_{i,j}^b$. Set all other positions to 0.
5. For $1 \leq i \leq m$, all parties P_i invoke $\mathcal{F}_{\text{ms}}$ with input $\vec{s}h_i$. P_i receives $\vec{s}h'_i$.
6. For $1 < j \leq m$, P_j sends $\vec{s}h'_j$ to P_1 .
7. P_1 recovers $\vec{v} = \bigoplus_{i=1}^m \vec{s}h'_i$ and sets $Y = \emptyset$. For $1 \leq i \leq (m-1)B$, if $v'_i = x \parallel H(x)$ holds for some $x \in \{0, 1\}^l$, adds x to Y . Outputs $X_1 \cup Y$.

Fig. 11. Our SK-MPSU $\Pi_{\text{SK-MPSU}}$

Proof. This proof is supposed to be divided into two cases in terms of whether $P_1 \in \text{Corr}$, since this determines whether the adversary has knowledge of the output. Nevertheless, the simulation of these two cases merely differ in the output reconstruction stage, thus we combine them together for the sake of simplicity. Specifically, the simulator receives the input X_c of $P_c \in \text{Corr}$ and the output $\bigcup_{i=1}^m X_i$ if $P_1 \in \text{Corr}$.

For each P_c , its view consists of its input X_c , outputs from $\mathcal{F}_{\text{bssPMT}}$, $\mathcal{F}_{\text{mss-rot}}$, output $\vec{s}h'_c$ from $\mathcal{F}_{\text{ms}}$ as its share, sampled values as shares of Δ for $\mathcal{F}_{\text{mss-rot}}$ and $m-1$ sets of shares $\{\vec{s}h'_i\}_{1 \leq i \leq m}$ (P_i 's output from $\mathcal{F}_{\text{ms}}$) from P_i if $c = 1$. The simulator emulates each P_c 's view by running the protocol honestly with the following changes:

- In step 2, it simulates uniform outputs $\{e_{c,j}^b\}_{c < j \leq m}$ and $\{e_{c,i}^b\}_{1 \leq i < c}$ from $\mathcal{F}_{\text{bssPMT}}$, on condition that $P_i, P_j \in \mathcal{H}$.
- In step 3, it simulates uniform outputs $\{r_{c,ji}^b\}_{c < j \leq m, 1 \leq i < j}$ from $\mathcal{F}_{\text{mss-rot}}$, on condition that $\exists \min(2, i) \leq d \leq j, P_d \in \mathcal{H}$. If $c \neq 1$, it simulates uniform $\{\Delta_{c,ji}^b\}_{c < j \leq m, 1 \leq i < j}$ as P_c 's random tapes.
- In step 4, it simulates uniformly output $\vec{s}h'_c$ from $\mathcal{F}_{\text{ms}}$.

Now we discuss the case when $P_1 \in \text{Corr}$. In step 4 and 5, it computes $Y = \bigcup_{i=1}^m X_i \setminus X_1$ and constructs $\vec{v} \in (\{0, 1\}^{l+\kappa})^{(m-1)B}$ as follows:

- For $\forall x_i \in Y$, $v_i = x_i \| \mathbf{H}(x)$, $1 \leq i \leq |Y|$.
- For $|Y| < i \leq (m-1)B$, samples $v_i \leftarrow \{0, 1\}^{l+\kappa}$.

Then it samples a random permutation $\pi : [(m-1)B] \rightarrow [(m-1)B]$ and computes $\vec{v}' = \pi(\vec{v})$. For $1 \leq i \leq m$, it samples share $\vec{s}h'_i \leftarrow (\{0, 1\}^{l+\kappa})^{(m-1)B}$, which satisfies $\bigoplus_{i=1}^m \vec{s}h'_i = \vec{v}'$ and is consistent with the previous sampled $\vec{s}h'_c$ for each corrupted P_c . Add all $\vec{s}h'_i$ to P_1 's view and $\vec{s}h'_{c'}$ to each corrupted $P_{c'}$'s view ($c' \neq 1$) as its output from $\mathcal{F}_{\text{ms}}$, respectively.

The changes of outputs from $\mathcal{F}_{\text{bssPMT}}$ and $\mathcal{F}_{\text{mss-rot}}$ have no impact on P_c 's view, for the following reasons. By the definition of $\mathcal{F}_{\text{bssPMT}}$, each output $e_{c,j}^b$ and $e_{c,i}^b$ from $\mathcal{F}_{\text{bssPMT}}$ is uniformly distributed as a secret-share between P_c and P_j , or P_i and P_c , where $P_i, P_j \in \mathcal{H}$. By the definition of $\mathcal{F}_{\text{mss-rot}}$, each output $r_{c,ji}^b$ from $\mathcal{F}_{\text{mss-rot}}$ is a secret-share of 0 among $P_{\min(2,i)}, \dots, P_j$ if $e_{i,j}^b \oplus e_{j,i}^b = 0$, or a secret-share of $\bigoplus_{d=2}^j \Delta_{d,ji}^b$ if $e_{i,j}^b \oplus e_{j,i}^b = 1$. Therefore, even if P_c colludes with others, $r_{c,ji}^b$ is still uniformly random from the perspective of adversary, since there always exists a party $P_d \in \mathcal{H}(\min(2, i) \leq d \leq j)$ holding one share.

It remains to demonstrate that the output $\vec{s}h'_c$ from $\mathcal{F}_{\text{ms}}(P_1 \notin \text{Corr})$ or all outputs $\{\vec{s}h'_i\}_{1 \leq i \leq m}$ from $\mathcal{F}_{\text{ms}}(P_1 \in \text{Corr})$ does not leak any other information except for the union. The former case is easier to tackle with. The output $\vec{s}h'_c$ is distributed as a secret-share among all parties, so it is uniformly distributed from the perspective of adversary.

We now proceed to explain the latter case. For all $1 < j \leq m$, consider an element $x \in X_j$ and x is placed in the b th bin by P_j . In the real protocol, if there

is no $X_i (1 \leq i < j)$ s.t. $x \in X_i$, then for all $1 \leq i < j$, $e_{i,j}^y \oplus e_{j,i}^y = 0$. By the $\mathcal{F}_{\text{mss-rot}}$ functionality in Figure 9, each $r_{d,j,i}^b$ is uniform in $\{0, 1\}^{l+\kappa}$ conditioned on $\bigoplus_{d=\min(2,i)}^j r_{d,j,i}^b = 0$. From the descriptions of the protocol, We derive that each $u_{d,j}^b$ is uniform in $\{0, 1\}^{l+\kappa}$ conditioned on $\bigoplus_{d=\min(2,i)}^j u_{d,j}^b = x \| \mathbf{H}(x)$, namely, they are additive shares of $x \| \mathbf{H}(x)$ among all parties. This is exactly identical to the simulation.

If there exists some $X_i (1 \leq i < j)$ s.t. $x \in X_i$, then $e_{i,j}^b \oplus e_{j,i}^b = 1$. By the $\mathcal{F}_{\text{mss-rot}}$ functionality in Figure 9, each $r_{d,j,i}^b$ is uniform conditioned on $\bigoplus_{d=\min(2,i)}^j r_{d,j,i}^b = \bigoplus_{j'=2}^j \Delta_{j',ji}^b$, where each $\Delta_{j',ji}^b$ is uniformly held by $P_{j'}$ ($1 < j' \leq j$). From the descriptions of the protocol, We derive that each $u_{d,j}^b$ is uniform conditioned on $\bigoplus_{d=\min(2,i)}^j u_{d,j}^b = x \| \mathbf{H}(x) \oplus \bigoplus_{j'=2}^j \Delta_{j',ji}^b \oplus r$, where r is the sum of remaining terms. Then, even if P_1 colludes with others, $\bigoplus_{d=\min(2,i)}^j u_{d,j}^b$ is still uniformly random from the perspective of adversary, since there always exists a party $P_{j'} \in \mathcal{H} (1 < j' \leq j)$ holding one uniform $\Delta_{j',ji}^b$ and independent of all honest parties' inputs. For all empty bins, $u_{d,j}^b$ is chosen uniformly random, so the corresponding $\bigoplus_{d=1}^j u_{d,j}^b$ is also uniformly at random, which is identical to the simulation. By the definition of $\mathcal{F}_{\text{ms}}$, all parties additively share $\bigoplus_{d=1}^j u_{d,j}^b$ in a random permutation that maintains privacy against a coalition of arbitrary corrupted parties, and receive back $\{\vec{s}h_i'\}$, respectively. We conclude that all outputs $\{\vec{s}h_i'\}_{1 \leq i \leq m}$ from $\mathcal{F}_{\text{ms}}$ distribute identically between the real and ideal executions.

6 MPSU from Public-Key Techniques

In this section, we describe how to construct PK-MPSU from batch ssPMT and MKR-PKE. The construction follows the encryption based framework outlined in Section 2.4 and is formally presented in Figure 12.

In the first phase, each P_j ($2 \leq j \leq m$) first perform the replacing procedure with each P_i ($1 < i < j$): (1) P_j assigns X_j into B bins via Cuckoo hashing. P_i assigns X_i into B bins via simple hashing. P_j acts as $\mathcal{R}$ and executes batch ssPMT with P_i . For each bin, P_j and P_i receive shares $e_{j,i}$ and $e_{i,j}$. (2) If $i = 2$, P_j encrypts x using MKR-PKE as the initial value of m_0 and encrypts a dummy message using MKR-PKE as the initial value of m_1 , where $x \in X_j$ is the element in P_j 's bin. (3) P_j acts as $\mathcal{S}$ and executes two-choice-bit OT with P_i , where P_j and P_i input $e_{j,i}$ and $e_{i,j}$ as choice bits respectively (The two-choice-bit OT is identical to the standard 1-out-of-2 ROT, where the choice bit of $\mathcal{S}$ indicates whether to swap the order of m_0 and m_1). P_j inputs m_0, m_1 and P_i receives $\text{ct} = m_{e_{j,i} \oplus e_{i,j}}$. If $x \notin X_i$, ct is the ciphertext of x ; otherwise, the ciphertext of dummy message. (4) P_i rerandomizes ct to ct' and returns ct' to P_j . (5) P_j rerandomizes ct' , updates m_0 to ct' , and rerandomizes m_1 before repeating the procedure with the next party P_{i+1} . After interacting with all P_i ($1 < i < j$), P_j retains an encrypted x in the final m_0 , only if $x \notin X_i$ holds for all i . As a result, P_j holds the encrypted elements in $X_j \setminus (X_2 \cup \dots \cup X_{j-1})$.

Parameters. m parties $P_1, \dots, P_m$. Size n of input sets. The bit length l of set elements. Cuckoo hashing parameters: hash functions h_1, h_2, h_3 and number of bins B . A MKR-PKE scheme $\mathcal{E} = (\text{Gen}, \text{Enc}, \text{ParDec}, \text{Dec}, \text{ReRand})$.

Inputs. Each party P_i has input $X_i = \{x_i^1, \dots, x_i^n\} \subseteq \{0, 1\}^l$.

Protocol.

0. For $1 \leq i \leq m$, P_i runs $(pk_i, sk_i) \leftarrow \text{Gen}(1^\lambda)$, and distributes its public key pk_i to other parties. Define $sk = sk_1 + \dots + sk_m$ and all parties can compute the associated public key $pk = \prod_{i=1}^m pk_i$.
1. P_1 does $\mathcal{T}_1^1, \dots, \mathcal{T}_1^B \leftarrow \text{Simple}_{h_1, h_2, h_3}^B(X_1)$. For $1 < j \leq m$, P_j does $\mathcal{C}_j^1, \dots, \mathcal{C}_j^B \leftarrow \text{Cuckoo}_{h_1, h_2, h_3}^B(X_j)$ and $\mathcal{T}_j^1, \dots, \mathcal{T}_j^B \leftarrow \text{Simple}_{h_1, h_2, h_3}^B(X_j)$.
2. For $1 < j \leq m$:
 - For $1 \leq i < j$: P_i and P_j invoke $\mathcal{F}_{\text{bssPMT}}$ of batch size B , where P_i acts as sender with inputs $\mathcal{T}_i^1, \dots, \mathcal{T}_i^B$ and P_j acts as receiver with inputs $\mathcal{C}_j^1, \dots, \mathcal{C}_j^B$. For the instance $b \in [B]$, P_i receives $e_{i,j}^b \in \{0, 1\}$, and P_j receives $e_{j,i}^b \in \{0, 1\}$.
3. For $1 < j \leq m$, for each bin $b \in [B]$:
 - P_j defines $\tilde{c}_j$ and sets $c_j^b = \text{Enc}(pk, \text{Elem}(\mathcal{C}_j^b))$. $\text{Elem}(\mathcal{C}_j^b)$ denotes the element in $\mathcal{C}_j^b$.
 - For $1 < i < j$:
 - P_i and P_j invoke $\mathcal{F}_{\text{rot}}$ where P_i acts as receiver with input $e_{i,j}^b$ and P_j acts as sender. P_i receives $r_{i,j}^b = r_{j,i,e_{i,j}^b}^b \in \{0, 1\}^\lambda$. P_j receives $r_{j,i,0}^b, r_{j,i,1}^b \in \{0, 1\}^\lambda$. P_j computes $u_{j,i,e_{j,i}^b}^b = r_{j,i,e_{j,i}^b}^b \oplus c_j^b$, $u_{j,i,e_{j,i}^b \oplus 1}^b = r_{j,i,e_{j,i}^b \oplus 1}^b \oplus \text{Enc}(pk, \perp)$, then sends $u_{j,i,0}^b, u_{j,i,1}^b$ to P_i .
 - P_i defines $v_{j,i}^b = u_{j,i,e_{i,j}^b}^b \oplus r_{i,j}^b$ and sends $v_{j,i}^b = \text{ReRand}(pk, v_{j,i}^b)$ to P_j . P_j updates $c_j^b = \text{ReRand}(pk, v_{j,i}^b)$.
4. For $1 < j \leq m$, for each bin $b \in [B]$:
 - P_1 and P_j invoke $\mathcal{F}_{\text{rot}}$ where P_1 acts as receiver with input $e_{1,j}^b$ and P_j acts as sender. P_1 receives $r_{1,j}^b = r_{j,1,e_{1,j}^b}^b \in \{0, 1\}^\lambda$. P_j receives $r_{j,1,0}^b, r_{j,1,1}^b \in \{0, 1\}^\lambda$. P_j computes $u_{j,1,e_{j,1}^b}^b = r_{j,1,e_{j,1}^b}^b \oplus c_j^b$, $u_{j,1,e_{j,1}^b \oplus 1}^b = r_{j,1,e_{j,1}^b \oplus 1}^b \oplus \text{Enc}(pk, \perp)$, then sends $u_{j,1,0}^b, u_{j,1,1}^b$ to P_1 .
 - P_1 defines $\tilde{\text{ct}}_1' \in (\{0, 1\}^\lambda)^{(m-1)B}$, and sets $\text{ct}_1'^{(j-2)B+b} = \text{ReRand}(pk, u_{j,1,e_{1,j}^b}^b \oplus r_{1,j}^b)$.
5. P_1 samples $\pi_1 : [(m-1)B] \rightarrow [(m-1)B]$ and computes $\tilde{\text{ct}}_1'' = \pi_1(\tilde{\text{ct}}_1')$. P_1 sends $\tilde{\text{ct}}_1''$ to P_2 .
6. For $1 < j \leq m$:
 - For $1 \leq i \leq (m-1)B$: P_j computes $\text{ct}_j^i = \text{ParDec}(sk_j, \text{ct}_{j-1}''^i)$, $pk_{A_j} = pk_1 \cdot \prod_{d=j+1}^m pk_d$, and $\text{ct}_j^i = \text{ReRand}(pk_{A_j}, \text{ct}_j^i)$.
 - P_j samples $\pi_j : [(m-1)B] \rightarrow [(m-1)B]$. P_j defines $\tilde{\text{ct}}_j' = (\text{ct}_j^1, \dots, \text{ct}_j^{(m-1)B})$ and computes $\tilde{\text{ct}}_j'' = \pi_j(\tilde{\text{ct}}_j')$. If $j \neq m$, P_j sends $\tilde{\text{ct}}_j''$ to P_{j+1} ; else, P_m sends $\tilde{\text{ct}}_m''$ to P_1 .
7. P_1 sets $Y = \emptyset$. For $1 \leq i \leq (m-1)B$, P_1 computes $\text{pt}_i = \text{Dec}(sk_1, \text{ct}_m''^i)$. If $\text{pt}_i \neq \perp$, update $Y = Y \cup \{\text{pt}_i\}$. Output Y .

Fig. 12. Our PK-MPSU $\Pi_{\text{PK-MPSU}}$

To enable P_1 to finally obtain the encrypted set $X_j \setminus (X_1 \cup \dots \cup X_{j-1})$, P_j and P_1 engage in a similar procedure: After P_1 preprocessing X_1 via simple hashing, P_j and P_1 execute batch ssPMT and receive secret shares as choice bits for the subsequent executions of two-choice-bit OT. For each bin, P_i acts as $\mathcal{S}$ and inputs the final m_0 of the previous procedure and an encrypted dummy message. If $x \notin X_1$, P_1 receives the final m_0 ; otherwise, it receives the encrypted dummy message. This removes the elements in X_1 from the encrypted set $X_j \setminus (X_2 \cup \dots \cup X_{j-1})$ and lets P_1 obtain the resulting ciphertexts.

The second phase for collaborative decryption and shuffle works as follows: P_1 rerandomizes the ciphertexts from the first phase, permutes them using a random permutation π_1 , and sends these ciphertexts to P_2 . P_2 performs a partial decryption on the received ciphertexts, rerandomizes them, permutes them using a random permutation π_2 , and forwards these ciphertexts to P_3 . This iterative process continues until the last party, P_m , returns its permuted partially decrypted ciphertexts to P_1 . Finally, P_1 fully decrypts the ciphertexts, filters out the dummy elements, retains the set $X_1 \cup \dots \cup X_m \setminus X_1$, and appends the elements in X_1 to compute the union.

Theorem 4. *Protocol $\Pi_{\text{PK-MPSU}}$ securely implements $\mathcal{F}_{\text{mpsu}}$ against any semi-honest adversary corrupting $t < m$ parties in the $(\mathcal{F}_{\text{bssPMT}}, \mathcal{F}_{\text{rot}})$ -hybrid model, assuming the rerandomizable property and indistinguishable multiple encryptions (cf. Appendix B.1) of MKR-PKE scheme.*

Proof. The simulator receives the input X_c of $P_c \in \text{Corr}$ and the output $\bigcup_{i=1}^m X_i$ if $P_1 \in \text{Corr}$.

For each P_c , its view consists of its input X_c , outputs from $\mathcal{F}_{\text{bssPMT}}$ and $\mathcal{F}_{\text{rot}}$, protocol messages $\{u_{j,c,0}^b\}_{c < j \leq m}, \{u_{j,c,1}^b\}_{c < j \leq m}$ from P_j , rerandomization messages $\{v_{c,i}^b\}_{1 < i < c}$ from P_i , π_c , permuted partial decryption messages $\tilde{\text{ct}}''_{c-1}$ from P_{c-1} if $c > 1$, or $\tilde{\text{ct}}''_m$ from P_m if $c = 1$. The simulator emulates each P_c 's view by running the protocol honestly with the following changes:

- In step 2, it simulates uniform outputs $\{e_{c,j}^b\}_{c < j \leq m}$ and $\{e_{c,i}^b\}_{1 \leq i < c}$ from $\mathcal{F}_{\text{bssPMT}}$, on condition that $P_i, P_j \in \mathcal{H}$.
- In step 3, it simulates uniform outputs $\{r_{c,i,0}^b\}_{1 \leq i < c}, \{r_{c,i,1}^b\}_{1 \leq i < c}$ from $\mathcal{F}_{\text{rot}}$, and $\{r_{j,c,e_{c,j}^b}^b\}_{c < j \leq m}$ from $\mathcal{F}_{\text{rot}}$, on condition that $P_i, P_j \in \mathcal{H}$. For $c < j \leq m$, it computes $u_{j,c,e_{c,j}^b}^b = r_{j,c,e_{c,j}^b}^b \oplus \text{Enc}(pk, \perp)$ and simulates $u_{j,c,e_{c,j}^b \oplus 1}^b$ uniformly at random, on condition that $P_j \in \mathcal{H}$. For $1 < i < c$, it simulates $v_{c,i}^b = \text{Enc}(pk, \perp)$, on condition that $P_i \in \mathcal{H}$.
- If $P_1 \notin \text{Corr}$, in step 4, for $1 \leq i \leq (m-1)B$, it computes $\text{ct}_{c-1}^i = \text{Enc}(pk, \perp)$, and then simulates the vector $\tilde{\text{ct}}''_{c-1} = \pi(\tilde{\text{ct}}'_{c-1})$ from P_{c-1} , where π is sampled uniformly random and $P_{c-1} \in \mathcal{H}$.

Now we discuss the case when $P_1 \in \text{Corr}$. In step 4, assume d is the largest number that $P_d \in \mathcal{H}$, namely, $P_{d+1}, \dots, P_m \in \text{Corr}$. The simulator emulates the partial decryption messages $\tilde{\text{ct}}''_d$ from P_d in the view of P_{d+1} as follows:

- For $\forall x_i \in Y = \bigcup_{j=1}^m X_j$, $\text{ct}_d^i = \text{Enc}(pk_A, x_i)$, $1 \leq i \leq |Y|$.
- For $|Y| < i \leq (m-1)B$, sets $\text{ct}_d^i = \text{Enc}(pk_A, \perp)$.

where $pk_{A_d} = pk_1 \cdot \prod_{j=d+1}^m pk_j$. Then it samples a random permutation $\pi : [(m-1)B] \rightarrow [(m-1)B]$ and computes $\vec{\text{ct}}_d'' = \pi(\vec{\text{ct}}_d')$.

For other corrupted $P_{d'+1} \in \{P_2, \dots, P_{d-1}\}$, if $P_{d'} \in \mathcal{H}$, it simulates each partial decryption message $\text{ct}_{d'}^i = \text{Enc}(pk, \perp)$ for $1 \leq i \leq (m-1)B$, and then computes the vector $\vec{\text{ct}}_{d'}'' = \pi(\vec{\text{ct}}_{d'}')$ from $P_{d'}$, where $\pi_{d'}$ is sampled uniformly random. Append $\vec{\text{ct}}_{d'}''$ to the view of $P_{d'+1}$.

The changes of outputs from $\mathcal{F}_{\text{bssPMT}}$ and $\mathcal{F}_{\text{rot}}$ have no impact on P_c 's view, for similar reasons in Theorem 3.

Indeed, $u_{j,c,e_{c,j}^b \oplus 1}^b$ is uniform in the real process, as $r_{j,c,e_{c,j}^b \oplus 1}^b$ (which is one of P_j 's output from $\mathcal{F}_{\text{rot}}$ hidden from P_c , and is used to mask the encrypted message in $u_{j,c,e_{c,j}^b \oplus 1}^b$) is uniform and independent of $r_{j,c,e_{c,j}^b}^b$ from P_c 's perspective.

It's evident from the protocol specification and simulation that the simulated $u_{j,c,e_{c,j}^b}^b$ is identically distributed to that in the real process, conditioned on the event $e_{c,j}^b \oplus e_{j,c}^b = 1$. The analysis in the case of $e_{c,j}^b \oplus e_{j,c}^b = 0$ can be further divided into two subcases, $c \neq 1$ and $c = 1$. We first argue that when $c \neq 1$, $u_{j,c,e_{c,j}^b}^b$ emulated by simulator is indistinguishable from that in the real process.

In the real process, for $1 < c < j \leq m, 1 \leq b \leq B$, if $\text{Elem}(\mathcal{C}_j^b) \in X_j \setminus (X_2 \cup \dots \cup X_{c-1})$, $c_j^b = \text{Enc}(pk, \text{Elem}(\mathcal{C}_j^b))$, $u_{j,c,e_{c,j}^b}^b = r_{j,c,e_{c,j}^b}^b \oplus \text{Enc}(pk, \text{Elem}(\mathcal{C}_j^b))$; else $c_j^b = \text{Enc}(pk, \perp)$, $u_{j,c,e_{c,j}^b}^b = r_{j,c,e_{c,j}^b}^b \oplus \text{Enc}(pk, \perp)$. In the real process, for $1 < c < j \leq m, 1 \leq b \leq B$, $u_{j,c,e_{c,j}^b}^b = r_{j,c,e_{c,j}^b}^b \oplus \text{Enc}(pk, \perp)$. If there exists an algorithm that distinguishes these two process, it implies the existence of an algorithm that can distinguish two lists of encrypted messages, with no knowledge of sk (since sk is secret-shared among m parties, it is uniformly distributed for any coalition of $m-1$ parties). Consequently, this implies the existence of an adversary to break the indistinguishable multiple encryptions of $\mathcal{E}$ in Definition 2 (where $q = (m-1)B$).

When $c = 1$, $u_{j,e_{1,j}^b}^b$ emulated by simulator is indistinguishable from that in the real process for the similar reason as the above analysis when $c > 1$.

Next, we start demonstrating that all $v_{c,i}^b = \text{Enc}(pk, \perp)$ emulated by simulator are indistinguishable from the real ones via the sequences of hybrids:

- **Hyb₀** The real interaction. For $1 < i < c, 1 \leq b \leq B$: If $\text{Elem}(\mathcal{C}_c^b) \in X_c \setminus (X_1 \cup \dots \cup X_i)$, $v_{c,i}^b = \text{Enc}(pk, \text{Elem}(\mathcal{C}_c^b))$; else $v_{c,i}^b = \text{Enc}(pk, \perp)$. $v_{c,i}^b = \text{ReRand}(pk, v_{c,i}^b)$.
- **Hyb₁** For $1 < i < c, 1 \leq b \leq B$: If $\text{Elem}(\mathcal{C}_c^b) \in X_c \setminus (X_1 \cup \dots \cup X_i)$, $v_{c,i}^b = \text{Enc}(pk, \text{Elem}(\mathcal{C}_c^b))$; else $v_{c,i}^b = \text{Enc}(pk, \perp)$. This change is indistinguishable by the rerandomizable property of $\mathcal{E}$.
- **Hyb₂** For $1 < i < c, 1 \leq b \leq B$: $v_{c,i}^b = \text{Enc}(pk, \perp)$. This change is indistinguishable by the indistinguishable multiple encryptions of $\mathcal{E}$.

When $P_1 \in \text{Corr}$, we prove that $\vec{ct}_d''$ emulated by simulator is indistinguishable from that in the real process via the sequences of hybrids:

- **Hyb₀** The real interaction. $\vec{ct}_1'' = \pi_1(\vec{ct}_1')$. For $2 \leq j \leq d, 1 \leq i \leq (m-1)B$: $ct_j^i = \text{ParDec}(sk_j, ct_{j-1}^i), ct_j^i = \text{ReRand}(pk_{A_j}, ct_j^i), \vec{ct}_j'' = \pi_j(\vec{ct}_j')$.
- **Hyb₁** For $2 \leq j \leq d, 1 \leq i \leq (m-1)B$: $ct_j^i = \text{ParDec}(sk_j, ct_{j-1}^i)$. $\vec{ct}_d' = \text{ReRand}(pk_{A_d}, \vec{ct}_d), \vec{ct}_d'' = \pi(\vec{ct}_d')$, where $\pi = \pi_1 \circ \dots \circ \pi_d$. It's easy to see that **Hyb₁** is identical to **Hyb₀**.
- **Hyb₂** $\vec{ct}_1$ is replaced by the following:
 - For $\forall x_i \in Y = \bigcup_{j=1}^m X_j, ct_1^i = \text{Enc}(pk, x_i), 1 \leq i \leq |Y|$.
 - For $|Y| < i \leq (m-1)B$, sets $ct_1^i = \text{Enc}(pk, \perp)$.**Hyb₂** rearranges $\vec{ct}_1$ and it is identical to **Hyb₁** as the adversary is unaware of π_d s.t. the order of elements in $\vec{ct}_1$ has no effect on the result of $\vec{ct}_d''$.
- **Hyb₃** $\vec{ct}_d$ is replaced by the following:
 - For $\forall x_i \in Y = \bigcup_{j=1}^m X_j, ct_d^i = \text{Enc}(pk_{A_d}, x_i), 1 \leq i \leq |Y|$.
 - For $|Y| < i \leq (m-1)B$, sets $ct_d^i = \text{Enc}(pk_{A_d}, \perp)$.
 The indistinguishability between **Hyb₃** and **Hyb₂** is implied by the partially decryptable property of $\mathcal{E}$.
- **Hyb₄** $\vec{ct}_d'$ is replaced by the following:
 - For $\forall x_i \in Y = \bigcup_{j=1}^m X_j, ct_d^i = \text{Enc}(pk_{A_d}, x_i), 1 \leq i \leq |Y|$.
 - For $|Y| < i \leq (m-1)B$, sets $ct_d^i = \text{Enc}(pk_{A_d}, \perp)$.**Hyb₄** is indistinguishable to **Hyb₃** because of the rerandomizable property of $\mathcal{E}$.
- **Hyb₅** The only change in **Hyb₅** is that π are sampled uniformly by the simulator. **Hyb₅** generates the same $\vec{ct}_d''$ as in the simulation. Given that π_d is uniform in the adversary's perspective, the same holds for π , so **Hyb₅** is identical to **Hyb₄**.

When $P_1 \notin \text{Corr}$, the simulator is unaware of the final union, so it has to emulate the partial decryption messages $\vec{ct}_{c-1}''$ as permuted $\text{Enc}(pk, \perp)$ in the view of P_c . Compared to the above hybrid argument, we only need to add one additional hybrid after **Hyb₄** to replace all rerandomized partial decryption messages $\vec{ct}_{c-1}'$ with $\text{Enc}(pk, \perp)$. This change is indistinguishable by the indistinguishable multiple encryptions of $\mathcal{E}$, as the adversary cannot distinguish two lists of messages with no knowledge of the partial secret key $sk_{A_{c-1}} = sk_1 + sk_c + \dots + sk_m$ (it is unaware of sk_1).

The same applies for the simulation of the partial decryption messages $\vec{ct}_{d'}''$ in the view of corrupted $P_{d'+1}$ ($d' \neq d$) when $P_1 \in \text{Corr}$. $\vec{ct}_{d'}''$ is also emulated by permuted $\text{Enc}(pk, \perp)$. To avoid repetition, we omit the analysis here.

Remark 2. When instantiating our PK-MPSU framework with EC MKR-PKE, the element space is set as EC points accordingly, which might limit its usage in applications. We also identified the same issue in [28]. However, we argue that this limitation is not inherent in our PK-MPSU (nor to the protocol in [28]),

as it can be addressed by finding an elliptic curve with efficient encoding and decoding algorithms or by exploring alternative instantiations of MKR-PKE.

Despite the limitation, our EC-element-space PK-MPSU remains useful in many scenarios. For instance, we present the first multi-party private ID protocol in Appendix C, where our EC-element-space PK-MPSU MPSU serves as a core subprotocol. Our multi-party private-ID takes advantages of our PK-MPSU and achieves linear computation and communication complexity and low communication costs as well.

7 Theoretical Analyses and Comparisons

In this section, we first analyze the communication and computation complexity of our batch ssPMT, and present a theoretical comparison with the mq-ssPMT in LG. Then we analyze the communication and computation complexity of our SK-MPSU and PK-MPSU protocols, and present theoretical comparisons with LG and [28] respectively.

7.1 Batch ssPMT's Complexity

The costs of each stage in Π_{bssPMT} (Figure 8) are calculated as follows.

Batch OPRF. The cost of batch OPRF mainly consists of two parts:

- Batch OPRF: There are several options for instantiating batch OPRF functionality [38, 13]. We opt for subfield vector oblivious linear evaluation (subfield-VOLE) [9, 10, 53] to instantiate batch OPRF using the approach in [13].

For $B = O(n)$ instances of OPRF, we require performing a subfield-VOLE of size B in the offline phase, and sending B derandomization messages of length l in the online phase. We resort to *Dual LPN with Fixed Weight and Regular Noise* [9, 10] to improve efficiency of subfield-VOLE, and instantiate the code family with Expand-Convolute codes [53], which enables near-linear time syndrome computation. The computation complexity is $O(n \log n)$. The computation complexity is $O(t\lambda \log n/t)$, where t is the fixed noise weight⁷. The round complexity is $O(1)$.

We denote the output length of OPRF as γ . The lower bound of γ is relevant to the total number of the batch OPRF invocations. In our SK-MPSU and PK-MPSU, for $1 \leq i < j \leq m$, P_i and P_j invoke batch OPRF. Overall, there are $1 + 2 + \dots + (m - 1) = (m^2 - m)/2$ invocations of batch OPRF. Considering all these invocations, we set $\gamma \geq \sigma + \log((m^2 - m)/2) + \log_2 B$, so that the probability of any $t_i = s_i$ occurring if $x \notin X_i$, which is $((m^2 - m)/2)B \cdot 2^{-\gamma}$, is less than or equal to $2^{-\sigma}$.

⁷ For instance, $t \approx 128$.

- **OKVS:** The size of key-value pairs encoded into OKVS is $|K_1| + \dots + |K_B|$. Taking into account the invocation of batch ssPMT in our two MPSU protocols in advance, this size is $3n$. We use the OKVS construction of [52], and the computation complexities of Encode_H and Decode_H algorithms are both $O(n)$. As we employ their $w = 3$ scheme with a cluster size of 2^{14} , the size of OKVS is $1.28\gamma \cdot (3n)$ bits.
- **Offline phase.** The computation complexity per party is $O(n \log n)$. The communication complexity per party is $O(t\lambda \log(n/t))$. The round complexity is $O(1)$.
- **Online phase.** The computation complexity per party is $O(n)$. The communication complexity per party is $O(\gamma n)$. The round complexity is $O(1)$.

Secret-shared private equality tests. Like [49], we instantiate ssPEQT (Figure 4) using the generic MPC techniques. The circuit of $\mathcal{F}_{\text{ssPEQT}}$ is composed of $\gamma - 1$ AND gates in GMW [32], where the inputs are already in the form of secret-shaing. Executing $\gamma - 1$ AND gates in sequence would incur $\gamma - 1$ rounds. To reduce the round complexity, we leverage a divide-and-conquer strategy and recursively organize the AND gates within a binary tree structure, where each layer requires one round. This ensures that the number of rounds is directly related to the depth of the tree (i.e., $O(\log \gamma)$). To sum up, each party invoke B instances of ssPEQT, which amounts to $(\gamma - 1)B$ AND gates and takes $O(\log \gamma)$ rounds.

We use silent OT extension [9] to generate Beaver triples in offline phase, then each AND gate only requires 4 bits communication and $O(1)$ computation in the online phase. As a result, an invocation of ssPEQT requires 4γ bits communication and $O(1)$ computation in the online phase⁸.

- **Offline phase.** The computation complexity per party is $O(\gamma n \log n)$. The communication complexity per party is $O(t\lambda \log(\gamma n/t))$. The round complexity is $O(1)$.
- **Online phase.** The computation complexity per party is $O(n)$. The communication complexity per party is $O(\gamma n)$. The round complexity is $O(\log \gamma)$.

Comparison with mq-ssPMT. Liu and Gao adapt the SKE-based mq-RPMT in [58] to construct mq-ssPMT. The most expensive part of their construction is secret-shared oblivious decryption-then-matching (ssVODM) [58], which is to implement a decryption circuit and a comparison circuit by the GMW protocol [32]. In total, the ssVODM requires $(T + \gamma' - \log n - 1)n$ AND gates, where γ' is the bit length of SKE ciphertext, and T is the number of AND gates in the SKE decryption circuit and is set to be considerably large (≈ 600) according

⁸ When calculating computational complexity, one evaluation on PRG or one hash operation is usually considered as $O(1)$ operation. However, here, the computational unit is bitwise XOR operation, and it's evident that performing $O(\gamma)$ bitwise XOR operations is much faster than executing $O(1)$ PRG evaluation or hash operation. Therefore, we count the online computation complexity of ssPEQT as $O(1)$

to their paper. The lower bound of γ' is relevant to the total number of the mq-ssPMT invocations. In LG, for $1 \leq i < j \leq m$, P_i and P_j invoke mq-ssPMT. Overall, there are $1 + 2 + \dots + (m-1) = (m^2 - m)/2$ invocations of mq-ssPMT. Considering all these invocations, $\gamma' \geq \sigma + \log((m^2 - m)/2) + \log_2 n$, so that the error probability, which is $((m^2 - m)/2)n \cdot 2^{-\gamma'}$, is less than or equal to $2^{-\sigma}$. However, γ' is incorrectly set as the bit length of input elements in their paper.

The costs of batch ssPMT of size $B = 1.27n$ ⁹ consist of the costs of batch OPPRF of size $1.27n$ and the costs of $1.27n$ instances of ssPEQT, where batch OPPRF can be instantiated by extremely fast specialized protocol and ssPEQT is implemented by the GMW protocol. Moreover, the state-of-the-art batch OPPRF construction [16, 55, 52] can achieve linear computation and communication with respect to n . In ssPEQT, the parties engage in $1.27(\gamma - 1)n$ AND gates, where γ is the output length of OPPRF. In the typical setting, we have $(T + \gamma' - \log n - 1)n \gg 1.27(\gamma - 1)n$, which means that the number of AND gates desired involved in batch ssPMT is far smaller than mq-ssPMT. Therefore, the construction built on batch ssPMT greatly reduces the dependency on general 2PC and significantly decreases both computation and communication complexity by a considerable factor, compared to mq-ssPMT.

7.2 Our SK-MPSU's Complexity

The costs of each stage in $\Pi_{\text{SK-MPSU}}$ (Figure 11) are calculated as follows.

Batch ssPMT. To achieve linear communication of this stage, we use stash-less Cuckoo hashing [49]. To render the failure probability (failure is defined as the event where an item cannot be stored in the table and must be stored in the stash) less than 2^{-40} , we set $B = 1.27n = O(n)$ for 3-hash Cuckoo hashing. The cost of batch ssPMT follows the complexity analysis in Section 4. For $1 \leq i < j \leq m$, P_i and P_j invoke batch ssPMT. Overall, each party P_j engages in $m - 1$ invocations of batch ssPMT, acting as $\mathcal{R}$ in the first $j - 1$ invocations, and acting as $\mathcal{S}$ in the last $m - j$ invocations.

- **Offline phase.** The computation complexity per party is $O(\gamma mn \log n)$. The communication complexity per party is $O(t\lambda m \log(\gamma n/t))$. The round complexity is $O(1)$.
- **Online phase.** The computation complexity per party is $O(mn)$. The communication complexity per party is $O(\gamma mn)$. The round complexity is $O(\log \gamma)$.

Multi-party secret-shared random oblivious transfers. Each invocation of mss-ROT involves pairwise executions of two-party OT, where each OT execution consists of two parts: In the offline phase, the parties invoke random-choice-bit ROT, then $\mathcal{S}$ sends one messages of length $l + \kappa$ ($\kappa = \sigma + \log(m - 1) + \log n$) to $\mathcal{R}$; In the online phase, $\mathcal{R}$ sends a 1-bit derandomization message to transform

⁹ We use stash-less Cuckoo hashing [49] with 3 hash functions, where $B = 1.27n$.

ROT into a chosen-input version. If P_i and P_j hold choice bits, then P_i and P_j invoke OT with the parties in $\{P_2, \dots, P_j\}$ except itself.

For $1 \leq i < j \leq m$, $P_{\min(2,i)}, \dots, P_j$ engage in B instances of mss-ROT, where P_i and P_j hold the two choice bits. Considering the overall invocations of mss-ROT: P_1 invokes $[\sum_{j=2}^m (j-1)]B = \frac{m^2 B - mB}{2}$ instances of two-party OT; For $1 < i \leq m$, each party P_i invokes $[(i-1) + \sum_{j=i+1}^m 3(j-2)]B = \frac{3m^2 B - 3i^2 B - 9mB + 11iB - 2B}{2}$ instances of two-party OT.

- **Offline phase.** The computation complexity per party is $O(m^2 n \log n)$. The communication complexity per party is $O(t\lambda m^2 \log(n/t))$. The round complexity is $O(1)$.
- **Online phase.** The computation complexity per party is $O(m^2 n)$. The communication complexity per party is $O(m^2 n)$. The round complexity is $O(1)$.

Multi-party secret-shared shuffle. Eskandarian et al. [25] proposed an online-efficient multi-party secret-shared shuffle protocol by extending [17] to the multi-party setting. In the offline phase, each party generates a random permutation and a set of correlated vectors called share correlations. In the online phase, each party efficiently permutes and refreshes the share vectors using these share correlations.

We use the construction in [25]. In the offline phase, each pair of parties runs a Share Translation protocol [17] of size $(m-1)B$ and length $l + \kappa$.

- **Offline phase.** The computation complexity per party is $O(m^2 n \log(mn))$. The communication complexity per party is $O(\lambda m^2 n \log(mn))$. The round complexity is $O(1)$.
- **Online phase.** The computation complexity of P_1 is $O(m^2 n)$. The communication complexity of P_1 is $O((l + \kappa)m^2 n)$. The computation complexity of P_j is $O(mn)$. The communication complexity of P_j is $O((l + \kappa)mn)$. The round complexity is $O(m)$.

Output reconstruction. For $1 < j \leq m$, P_j sends $\vec{s}h'_j \in (\{0, 1\}^{l+\kappa})^{(m-1)B}$ to P_1 . P_1 reconstructs $(m-1)B$ secrets, each having m shares.

- **Online phase.** The computation complexity of P_1 is $O(m^2 n)$. The communication complexity of P_1 is $O((l + \kappa)m^2 n)$. The communication complexity of P_j is $O((l + \kappa)mn)$. The round complexity is $O(1)$.

Comparison with LG. We present a comparison of the theoretical computation and communication complexity for each party in both offline and online phases between LG and our SK-MPSU in Table 2.

7.3 Our PK-MPSU's Complexity

The costs of each stage in $\Pi_{\text{PK-MPSU}}$ (Figure 12) are calculated as follows.

	Computation			Communication		
	Offline	Online		Offline	Online	
		Leader	Client		Leader	Client
[42]	$(T + \gamma' + m)mn \log n$	$(T + \gamma' + m)mn$	$(T + \gamma')mn$	$\lambda m^2 n \log n$	$(T + \gamma')mn + lm^2 n$	$(T + \gamma')mn$
Ours	$(\gamma + m)mn \log n$	$m^2 n$		$\lambda m^2 n \log n$	$\gamma mn + lm^2 n$	$(\gamma + l + m)mn$

Table 2. Asymptotic communication (bits) and computation costs of LG and our SK-MPSU in the offline and online phases. n is the set size. m is the number of participants. λ and σ are computational and statistical security parameter respectively and $\lambda = 128$, $\sigma = 40$. T is the number of AND gate in an SKE decryption circuit, $T \approx 600$. l is the bit length of input elements. γ is the output length of OPPRF. γ' is the bit length of SKE ciphertext and $\gamma' \approx \gamma$. t is the noise weight in dual LPN, $t \approx 128$.

Batch ssPMT The cost of this stage is identical to the batch ssPMT stage in $\Pi_{\text{SK-MPSU}}$ (cf. Section 7.2).

- **Offline phase.** The computation complexity per party is $O(\gamma mn \log n)$. The communication complexity per party is $O(t\lambda m \log(\gamma n/t))$. The round complexity is $O(1)$.
- **Online phase.** The computation complexity per party is $O(mn)$. The communication complexity per party is $O(\gamma mn)$. The round complexity is $O(\log \gamma)$.

Random oblivious transfers and messages rerandomization. We use EC ElGamal encryption to instantiate the MKR-PKE scheme. So each of encryption, partial decryption and rerandomization takes one point scalar operation. The length of ciphertext is 4λ .

For $1 \leq i < j \leq m$, P_i and P_j invoke B instances of silent ROT correlations with random inputs during the offline phase. In the online phase, P_i sends 1-bit derandomization message to transform each ROT into a chosen-input version. For each OT correlation, each P_j executes two encryptions and sends two 4λ -bit messages to P_i . P_i executes one rerandomization. If $i \neq 1$, P_i sends one ciphertext to P_j and P_j executes one rerandomization as well.

- **Offline phase.** The computation complexity per party is $O(mn \log n)$. The communication complexity per party is $O(t\lambda m \log(n/t))$. The round complexity is $O(1)$.
- **Online phase.** The computation complexity per party is $O(mn)$ public-key operations. The communication complexity per party is $O(\lambda mn)$. The round complexity is $O(1)$.

Messages decryptions and shufflings. Each party shuffles $(m-1)B$ ciphertexts and executes $(m-1)B$ partial decryptions before sending them to the next party.

- **Online phase.** The computation complexity per party is $O(mn)$ public-key operations. The communication complexity per party is $O(\lambda mn)$. The round complexity is $O(m)$.

Comparison with [28]. In Table 3, we present a comparison of the theoretical computation and communication complexity for each party in both offline and online phases between [28] and our PK-MPSU.

We conclude that the complexity of our PK-MPSU surpasses theirs in all respects, including the computation and communication complexity of the leader and clients in the offline and online phases, as depicted in the Table 3.

	Computation		Communication	
	Offline	Online	Offline	Online
[28]	$\gamma mn \log n (\log n / \log \log n)$	$mn (\log n / \log \log n)$	$t \lambda m \log n (\log n / \log \log n)$	$(\gamma + \lambda) mn (\log n / \log \log n)$
Ours	$\gamma mn \log n$	mn	$t \lambda m \log n$	$(\gamma + \lambda) mn$

Table 3. Asymptotic communication (bits) and computation costs of [28] and our PK-MPSU in the offline and online phases. In the offline phase, the computation is composed of symmetric-key operations; In the online phase, the computation is composed of public-key operations since we ignore symmetric-key operations. n is the set size. m is the number of participants. λ and σ are computational and statistical security parameter respectively and $\lambda = 128$, $\sigma = 40$. l is the bit length of input elements. γ is the output length of OPRF. t is the noise weight in dual LPN, $t \approx 128$.

8 Performance Evaluation

In this section, we provide the implementation details and experimental results for our works. For most previous works [37, 27, 8, 28] do not provide open-source code, and the protocol in [57] shows fairly inferior performance in comparison with the state-of-the-art LG, here we only compare our protocols with LG, whose implementation is available on <https://github.com/lx-1234/MPSU>. Our implementation is available on <https://github.com/real-world-cryptography/MPSU>.

During our experiments, we identify several issues in the LG implementation:

1. The code is neither a complete nor a secure implementation for MPSU, as it lacks the offline share correlation generation of the multi-party secret-shared shuffle protocol [25]. Instead of adhering to the protocol specifications, the implementation designates one party to generate and store some “fake” share correlations [25] as local files during the offline phase, while the other parties read these files and consume the share correlations in the online phase (refer to the function `ShareCorrelation::generate()` in `ShareCorrelation-Gen.cpp`). This causes two serious consequences:
 - The distributed execution is not supported.
 - The security of multi-party secret-shared shuffle is compromised, leading to information leakage.

To conduct a fair and complete comparison, we integrated our correct version of the share correlation generation implementation into their code.

2. The code is not a correct implementation, as it produces incorrect results when the set size n increases beyond a certain threshold. We figure out that this issue arises from the use of a fixed parameter (the variable `batchsize` in line 30 of `circuit/TripleGen.cpp`) for generating Beaver Triples in the offline phase, which leads to an insufficient number of Beaver Triples for the online phase when n is large. We modified their code to ensure correct execution and mark the cases where the original code fails with `*`.

8.1 Experimental Setup

We conduct our experiments on an Alibaba Cloud virtual machine with Intel(R) Xeon(R) 2.70GHz CPU (32 physical cores) and 128 GB RAM. We emulate the network connections using Linux `tc` command. In the LAN setting, the bandwidth is set to be 10 Gbps with 0.1 ms RTT latency. In the WAN settings, we test on two different network bandwidths 400 Mbps and 50Mbps, with 80 ms RTT latency. We measure the running time as the maximal time from protocol begin to end, including messages transmission time, and the communication costs as the total data that leader sent and received. For a fair comparison, we stick to the following settings:

- We set the computational security parameter $\lambda = 128$ and the statistical security parameter $\sigma = 40$.
- We test the balanced scenario by setting all m input sets to be of equal size. In LG and our SK-MPSU, each party holds n 64-bit strings. In our PK-MPSU, each party holds n elements encoded as EC points in compressed form.
- Each party uses $m - 1$ threads to interact simultaneously with all other parties and 4 threads to perform share correlation generation (in LG and our SK-MPSU), Beaver Triple generation (in all three), parallel SKE encryption (in LG), ciphertext rerandomization and partial decryption (in our PK-MPSU).

8.2 Implementation Details

Our protocols are written in C++, and we use the following libraries in our implementation.

- VOLE: We use VOLE implemented in `libOTe` [54], instantiating the code family with Expand-Convolute codes [53].
- OKVS and GMW: We use the optimized OKVS construction in [52]¹⁰ and re-use the OKVS implementation in [5]. We also re-use the GMW implementation in [5] to construct ssPEQT.

¹⁰ Since the existence of suitable parameters for the new OKVS construction of the recent work [7] is unclear when the set size is less than 2^{10} , we choose to use the OKVS construction of [52].

- ROT: We use SoftSpokenOT [56] implemented in libOTe, and set field bits to 5 to balance computation and communication costs.
- Share Correlation: We re-use the implementation of Permute+Share [43, 17] in [4] to build share correlation generation for our SK-MPSU and LG.
- MKR-PKE: We implement MKR-PKE on top of the curve NIST P-256 (also known as `secp256r1` and `prime256v1`) implementation from `openssl` [3].
- Additionally, we use the `cryptoTools` [2] library to compute hash functions and PRNG calls, and we adopt `Coproto` [1] to realize network communication.

8.3 Experimental Results

We conduct extensive experiments across various numbers of parties $\{3, 4, 5, 7, 9, 10\}$ and a wide range of set sizes $\{2^6, 2^8, 2^{10}, 2^{12}, 2^{14}, 2^{16}, 2^{18}, 2^{20}\}$ in the LAN and WAN settings. The performance of protocols is evaluated from four dimensions: online and total running time, and online and total communication costs. The results for running time / communication costs are depicted in Table 4. As we can see in the table, our protocols outperform LG in all the case studies, even with enhanced security.

Online computation improvement. Our SK-MPSU achieves a $3.9 - 10.0\times$ speedup compared to LG in the LAN setting, and a $1.1 - 2.1\times$ / $1.1 - 2.6\times$ speedup compared to LG in the 400 / 50 Mbps network. For example, in the online phase to compute the union of 2^{20} -size sets among 3 parties, our SK-MPSU runs in 4.4 seconds (LAN) and 33.1 / 121.1 seconds (400 / 50 Mbps), which is $5.8\times$ and $1.64\times$ / $1.94\times$ faster than LG that takes 25.2 seconds (LAN) and 54.1 / 234.4 seconds (400 / 50 Mbps), respectively.

Total computation improvement. Our SK-MPSU achieves a $1.2 - 7.8\times$ speedup compared to LG in the LAN setting. Our PK-MPSU achieves a $1.2 - 4.0\times$ / $1.8 - 9.2\times$ speedup compared to LG in the 400 / 50 Mbps network. For example, to compute the union of 2^{20} -size sets among 3 parties, our SK-MPSU runs in 96.2 seconds overall (LAN), which is $6.4\times$ faster than LG that takes 610.8 seconds. To compute the union of 2^{18} -size sets among 9 parties, our PK-MPSU runs in 38.4 / 47.2 minutes overall (400 / 50 Mbps), which is $2.8\times$ / $8.5\times$ faster than LG that takes 106.4 / 396.9 minutes.

Online communication improvement. The online communication costs of our SK-MPSU is $1.2 - 4.9\times$ smaller than LG. For example, in the online phase to compute the union of 2^{20} -size sets among 3 parties, our SK-MPSU requires 455.6 MB communication, which is a $2.0\times$ improvement of LG that requires 917.0 MB communication.

Total communication improvement. The total communication costs of our PK-MPSU is $3.0 - 36.5\times$ smaller than LG. For example, to compute the union of 2^{18} -size sets among 9 parties, our PK-MPSU requires 960.1 MB communication, a $34.8\times$ improvement of LG that requires 33360 MB.

In conclusion, our protocols exhibit distinct advantages and are suitable for different scenarios:

Sett.	m	Protocol	Set size n																
			Online								Total = Offline + Online								
			2 ⁶	2 ⁸	2 ¹⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰	2 ⁶	2 ⁸	2 ¹⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰	
Time. (s) LAN	3	LG	0.050	0.058	0.069	0.143	0.425	1.582	6.219	25.16*	0.982	1.016	1.098	1.767	5.650	32.42	153.8	610.8*	
		Our SK	0.005	0.007	0.009	0.017	0.050	0.213	1.005	4.352	0.361	0.386	0.395	0.489	1.170	4.534	19.77	96.23	
		Our PK	0.092	0.237	0.822	3.176	12.69	51.17	205.7	829.1	0.117	0.273	0.868	3.270	13.08	53.05	215.0	872.6	
	4	LG	0.069	0.076	0.093	0.162	0.478	1.637	6.375	25.79*	1.292	1.358	1.514	2.432	8.716	45.37	197.2	762.9*	
		Our SK	0.008	0.010	0.013	0.023	0.071	0.286	1.393	5.645	0.639	0.665	0.696	0.917	2.597	10.94	50.25	237.8	
		Our PK	0.159	0.465	1.570	6.136	24.60	44.91	398.4	1604	0.196	0.505	1.643	6.299	25.45	48.95	417.5	1694	
	5	LG	0.107	0.109	0.126	0.190	0.541	1.998	7.588	31.33*	1.939	1.993	2.202	3.408	12.54	65.04	289.9	1152*	
		Our SK	0.012	0.013	0.017	0.030	0.087	0.368	1.714	7.003	0.914	0.964	1.023	1.558	4.551	18.71	85.79	373.4	
		Our PK	0.232	0.693	2.487	9.858	39.33	158.1	637.5	2816	0.292	0.796	2.547	10.09	40.25	162.2	655.0	2979	
	7	LG	0.154	0.165	0.174	0.281	0.795	2.894	10.92	—	3.484	3.583	3.921	5.84	23.42	111.2	484.5	—	
		Our SK	0.019	0.021	0.028	0.048	0.156	0.607	2.817	—	2.051	2.079	2.214	3.470	12.92	56.50	245.6	—	
		Our PK	0.463	1.365	5.030	19.43	77.22	310.0	1247	—	0.550	1.469	5.158	19.73	78.59	316.3	1276	—	
Time. (s) 400Mbps	9	LG	0.222	0.226	0.234	0.417	1.216	4.449	17.29	—	5.501	5.612	6.249	10.27	41.69	182.7	793.3	—	
		Our SK	0.027	0.031	0.039	0.075	0.230	0.970	4.293	—	3.363	3.402	3.895	7.161	30.09	126.3	678.8	—	
		Our PK	0.764	2.271	8.074	31.64	126.7	507.0	2039	—	0.880	2.375	8.238	32.19	128.7	515.5	2080	—	
	10	LG	0.228	0.243	0.276	0.514	1.479	5.467	—	—	6.736	6.846	8.335	13.56	55.58	238.3	—	—	
		Our SK	0.031	0.036	0.043	0.088	0.286	1.183	—	—	3.965	4.232	4.821	9.334	41.97	175.9	—	—	
		Our PK	0.865	2.800	9.944	39.09	155.9	622.7	2503	—	0.970	2.912	10.11	39.58	158.4	634.6	2556	—	
	3	LG	4.502	4.505	4.522	4.914	6.272	8.744	17.78	54.13*	12.46	13.59	15.76	19.52	30.73	78.74	282.8	1188*	
		Our SK	2.165	2.166	2.332	3.157	3.734	4.444	9.705	33.10	8.403	9.710	12.62	16.30	25.02	56.88	194.7	801.2	
		Our PK	4.419	4.555	5.553	7.984	18.21	59.77	226.3	900.3	5.168	5.306	6.472	8.968	19.65	62.73	237.3	946.1	
	Time. (s) 50Mbps	4	LG	5.696	5.880	6.540	7.094	7.323	11.36	23.13	78.81*	17.94	20.99	27.63	32.69	50.59	145.7	554.0	2167*
			Our SK	2.967	2.969	3.298	3.976	4.618	6.507	17.10	59.21	11.46	13.93	20.21	26.77	47.49	133.2	520.0	2141
			Our PK	5.622	5.899	7.929	12.17	32.40	113.8	440.9	1761	6.773	7.052	9.25	13.53	34.27	118.9	461.3	1857
5		LG	7.385	7.708	8.621	9.198	9.687	14.62	32.83	126.3*	23.64	26.82	37.39	48.48	88.32	263.3	1053	4394*	
		Our SK	3.768	3.733	4.471	4.800	5.521	8.938	25.95	95.40	17.53	21.10	30.28	44.30	88.15	278.9	1119	4820	
		Our PK	6.849	7.928	10.50	17.08	49.75	179.8	703.3	2873	8.424	9.483	12.22	18.86	52.07	185.4	724.5	2980	
7		LG	9.312	9.833	10.55	11.35	12.14	21.29	66.93	—	34.92	45.69	66.26	94.89	203.4	705.8	2898	—	
		Our SK	5.373	5.381	6.207	6.644	8.164	17.67	56.894	—	34.00	44.95	61.34	95.03	228.7	817.4	3506	—	
		Our PK	9.504	12.32	15.14	29.73	92.66	348.5	1377.4	—	11.88	14.71	17.72	32.35	95.86	356.5	1409	—	
9		LG	11.41	12.21	13.34	14.41	15.09	33.84	115.5	—	56.65	75.24	104.1	169.1	406.0	1503	6387	—	
		Our SK	6.977	7.068	7.830	8.387	12.24	29.20	105.7	—	58.81	84.80	107.6	182.0	502.5	1915	8137	—	
		Our PK	13.67	18.84	22.96	45.54	148.2	570.9	—	—	16.87	22.04	26.31	48.98	152.3	581.5	2034	—	
Comm. (MB)	10	LG	11.77	12.24	15.80	16.49	17.48	45.20	—	—	66.19	92.22	125.1	219.8	582.1	2179	—	—	
		Our SK	7.780	8.032	8.635	9.203	14.54	38.31	—	—	71.58	109.2	132.7	242.7	684.0	2687	—	—	
		Our PK	16.32	23.05	28.66	59.79	184.8	708.5	2792	—	19.90	26.66	32.44	63.66	189.3	720.9	2846	—	
	3	LG	4.596	4.736	4.834	5.287	7.476	19.29	60.58	234.4*	13.05	14.54	18.58	25.31	46.49	149.4	610.0	2549*	
		Our SK	2.169	2.339	2.379	3.333	4.537	9.940	31.60	121.1	8.689	10.33	13.38	20.15	46.80	148.6	611.4	2766	
		Our PK	4.849	5.004	6.001	8.936	21.35	71.57	322.7	1120	5.603	5.762	6.929	9.904	22.71	74.20	332.1	1163	
	4	LG	5.653	5.900	6.205	6.932	11.88	32.10	119.4	470.2*	18.05	21.43	29.09	42.78	102.2	367.5	1544	6812*	
		Our SK	3.131	3.147	3.372	4.271	6.979	18.83	69.46	269.7	12.26	15.16	22.27	39.99	110.1	411.1	1768	8078	
		Our PK	6.393	6.664	8.348	14.20	38.71	138.4	536.9	2136	7.549	7.832	9.683	15.57	40.48	142.76	554.6	2215	
	5	LG	7.183	7.916	8.283	10.51	15.59	51.78	198.8	—	24.56	29.00	40.27	72.24	194.9	763.6	3308	—	
		Our SK	3.859	3.961	4.587	5.462	10.46	32.89	127.7	—	18.60	23.31	36.40	75.50	236.7	945.9	4105	—	
		Our PK	7.966	9.237	11.69	20.91	60.92	220.9	864.3	—	9.523	10.82	13.45	22.69	63.12	225.8	864.9	—	
7	LG	9.08	9.415	10.68	14.18	29.47	108.4	418.0	—	38.05	51.56	88.41	182.2	585.0	2423	10444	—		
	Our SK	5.471	5.594	6.468	8.883	22.92	80.96	318.9	—	36.57	49.54	83.742	201.8	714.3	2962	13265	—		
	Our PK	12.87	14.95	19.03	38.14	115.5	429.6	1693	—	15.25	17.38	21.60	40.75	118.6	437.0	1721	—		
9	LG	10.39	10.55	12.76	17.29	48.80	184.8	718.5	—	66.62	90.76	156.9	371.7	1314	5504	23814	—		
	Our SK	7.094	7.23	8.429	14.12	42.34	160.3	636.8	—	62.51	95.48	159.0	452.3	1708	7294	32303	—		
	Our PK	19.41	25.02	30.69	61.53	189.8	704.8	2793	—	22.67	28.34	34.13	65.06	193.8	714.4	2831	—		
10	LG	11.04	11.37	15.05	20.73	60.73	230.4	—	—	81.15	124.3	208.7	530.5	1921	8083	—	—		
	Our SK	7.826	8.222	9.636	17.56	55.45	214.5	—	—	76.53	122.1	211.7	620.8	2415	10418	—	—		
	Our PK	23.9	28.84	37.78	77.75	232.7	866.9	3440	—	27.61	32.53	41.67	81.74	237.2	878.9	3489	—		
Comm. (MB)	3	LG	0.157	0.284	0.962	3.662	14.43	57.58	229.8	917.0*	6.311	7.904	13.37	32.58	114.6	474.3	2052	8973*	
		Our SK	0.032	0.111	0.426	1.690	6.788	27.87	112.7	455.6	2.542	3.582	8.529	30.91	132.7	588.8	2614	11529	
		Our PK	1.815	1.983	2.655	5.418	16.36	60.78	239.3	959.5	2.084	2.325	3.073	5.908	16.92	61.41	240.0	960.3	
	4	LG	0.242	0.449	1.536	5.868	23.15	92.38	368.6	1471*	9.591	12.44	23.89	67.25	253.7	1074	4674	20419*	
		Our SK	0.058	0.204	0.791	3.145	12.81	51.69	208.8	843.7	3.941	6.385	16.96	66.89	293.9	1312	5834	25751	
		Our PK	2.723	2.975	3.983	8.126	24.54	101.9	359.0	1439	3.127	3.488	4.610	8.861	25.38	102.8	360.1	1440	
	5	LG	0.330	0.630	2.173	8.325	32.86	131.2	523.5	2090*	12.92	17.75	36.48	110.7	435.2	1868	8228	35737*	
		Our SK	0.090	0.323	1.257	5.007	20.36	82.11	331.5	1339	5.504	10.18	30.44	124.7	548.9	2445	10832	47635	
		Our PK	3.630	3.967	5.311	10.84	32.72	121.6	478.7	1919	4.169	4.650	6.147	11.82	33.84	122.8	480.1	1921	
	7	LG	0.519	1.038	3.635	13.99	55.29	220.8	881.3	—	19.92	29.00	41.59	243.3	997.9	4286	18924	—	
		Our SK	0.172	0.634	2.489	10.03	40.31	162.5	655.4	—	8.827	18.80	64.76	275.4	1226	5474	24275	—	
		Our PK	5.445	5.950	7.966	16.25	49.07	182.3	718.0	—	6.253	6.98	9.220	17.72	50.76	184.2	720.1	—	
9	LG	0.723	1.509	5.347	20.65	81.72	326.3	1303	—	28.15	44.74	115.0	413.3	1742	7609	33360	—		
	Our SK	0.279	1.046	4.122	16.60	66.76	269.0	1084	—	13.99	33.04	119.9	516.3	2295	12007	45609	—		
	Our PK	2.760	7.933	10.62	21.67	65.43	243.1	957.3	—	8.338	9.300	12.29	23.63	67.68	245.0	960.1	—		
10	LG	0.831	1.768	6.296	24.36	96.43	385.1	—	—	32.32	54.34	148.7	549.4	2314	10081	—	—		
	Our SK	0.341	1.288	5.086	20.48	82.39	332.0	—	—	16.21	40.56	149.9	551.6	2901	12908	—	—		
	Our PK	8.168	8.925																

- Our SK-MPSU: This protocol has the best online performance in both LAN and WAN settings. Notably, the online phase takes only 4.4 seconds for 3 parties with sets of 2^{20} items each, and 7 seconds for 5 parties with sets of 2^{20} items each in the LAN setting. In the WAN settings, it achieves a larger improvement over LG on slower network (50 Mbps), due to its lower online communication cost. Additionally, it has the best total performance in nearly all cases in the LAN setting. Therefore, our SK-MPSU is the optimal choice when online performance is the primary concern, or taking total performance into consideration in high-bandwidth networks.
- Our PK-MPSU: This protocol has the best total performance in the WAN settings, with a more significant improvement over LG in the slower network (50 Mbps), due to its lower total communication cost. Notably, in the 50 Mbps network, LG takes approximately 6.6 hours to run the entire protocol for 9 parties with sets of 2^{18} items each, whereas our PK-MPSU only takes 47 minutes. This demonstrates that our PK-MPSU particularly excels in bandwidth-constrained networks. Furthermore, our PK-MPSU is the only protocol with the capability of running among 10 parties with sets of 2^{18} items each in our experiments, which sufficiently indicates the superiority of its linear complexities.

Analysis of memory issues. In the offline phase, generating share correlations or Beaver Triples requires substantial memory when emulating all parties on a single machine for large m and n in both our protocols and LG. This leads to memory exhaustion in some trials, as indicated in Table 4. Take the share correlation generation in the offline phase of our SK-MPSU as an example: when $m = 7, n = 2^{20}$, each pair of parties executes the Permute+Share with $N = (m - 1) * 1.27n$ shares of 128 bits each. The communication cost is at least $C = 128 * N \log_2(N) = 128 * (m - 1) * 1.27n * \log_2((m - 1) * 1.27n)$ bits, approximately 2.73GB per pair. With $m * (m - 1)$ pairs of parties, the total memory allocated for data sending and receiving is $m * (m - 1) * C$, roughly 115GB. Considering other intermediate variables in the code, the total memory usage exceeds the 128GB capacity of our machine.

Acknowledgments

We thank the anonymous reviewers for their helpful comments. We also thank Jiahui Gao for the clarification of their work. This work is supported by the National Natural Science Foundation of China (No. 62272269 and No. 62402272), the National Key Research and Development Program of China (No. 2021YFA1000600), Taishan Scholar Program of Shandong Province, Shandong Laboratories Project of Bureau of Science & Technology of Shandong Province (SYS202201), and Key Project of Quan Cheng Laboratory (QCLZD202302).

References

1. Coproto: C++ coroutine protocol library. <https://github.com/Visa-Research/coproto.git>.
2. cryptoTools. <https://github.com/ladnir/cryptoTools.git>.
3. OpenSSL: TLS/SSL and crypto library. <https://github.com/openssl/openssl.git>.
4. PSU. <https://github.com/dujiajun/PSU.git>.
5. Vole-PSI. <https://github.com/Visa-Research/volepsi.git>.
6. Aner Ben-Efraim, Olga Nissenbaum, Eran Omri, and Anat Paskin-Cherniavsky. Psimple: Practical multiparty maliciously-secure private set intersection. In *ASIA CCS 2022*, pages 1098–1112. ACM, 2022.
7. Alexander Bienstock, Sarvar Patel, Joon Young Seo, and Kevin Yeo. Near-Optimal oblivious Key-Value stores for efficient PSI, PSU and Volume-Hiding Multi-Maps. In *USENIX Security 2023*, pages 301–318, 2023.
8. Marina Blanton and Everaldo Aguiar. Private and oblivious set and multiset operations. In *7th ACM Symposium on Information, Computer and Communications Security, ASIACCS 2012*, pages 40–41. ACM, 2012.
9. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. Efficient two-round OT extension and silent non-interactive secure computation. In *CCS 2019*, pages 291–308. ACM, 2019.
10. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In *CRYPTO 2019*. Springer, 2019.
11. Justin Brickell and Vitaly Shmatikov. Privacy-Preserving graph algorithms in the semi-honest model. In *ASIACRYPT 2005*, pages 236–252. Springer, 2005.
12. Prasad Buddhavarapu, Andrew Knox, Payman Mohassel, Shubho Sengupta, Erik Taubeneck, and Vlad Vlaskin. Private matching for compute. 2020. <https://eprint.iacr.org/2020/599>.
13. Dung Bui and Geoffroy Couteau. Improved private set intersection for sets with small entries. In *PKC 2023*, pages 190–220. Springer, 2023.
14. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *FOCS 2001*, pages 136–145. IEEE Computer Society, 2001.
15. Nishanth Chandran, Nishka Dasgupta, Divya Gupta, Sai Lakshmi Bhavana Obbattu, Sruthi Sekar, and Akash Shah. Efficient linear multiparty PSI and extensions to circuit/quorum PSI. In *CCS ’21*. ACM, 2021.
16. Nishanth Chandran, Divya Gupta, and Akash Shah. Circuit-PSI with linear complexity via relaxed batch OPPRF. *Proc. Priv. Enhancing Technol.*, 2022.
17. Melissa Chase, Esha Ghosh, and Oxana Poburinnaya. Secret-shared shuffle. In *Advances in Cryptology - ASIACRYPT 2020*, pages 342–372. Springer, 2020.
18. Melissa Chase and Peihan Miao. Private set intersection in the internet setting from lightweight oblivious PRF. In *CRYPTO 2020*, pages 34–63. Springer, 2020.
19. Yu Chen, Min Zhang, Cong Zhang, Minglang Dong, and Weiran Liu. Private set operations from multi-query reverse private membership test. In *Public-Key Cryptography - PKC 2024*. Springer, 2024.
20. Yu Chen, Min Zhang, Cong Zhang, Minglang Dong, and Weiran Liu. Private set operations from multi-query reverse private membership test. In *PKC 2024*, 2024. <https://eprint.iacr.org/2022/652>.
21. Michele Ciampi and Claudio Orlandi. Combining private set-intersection with secure two-party computation. In *Security and Cryptography for Networks, SCN 2018*, pages 464–482. Springer, 2018.

22. Ivan Damgård, Marcel Keller, Enrique Larraia, Valerio Pastro, Peter Scholl, and Nigel P. Smart. Practical covertly secure MPC for dishonest majority - or: Breaking the SPDZ limits. In *ESORICS 2013*. Springer, 2013.
23. Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *CRYPTO 2012*, pages 643–662. Springer, 2012.
24. Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31:469–472, 1985.
25. Saba Eskandarian and Dan Boneh. Clarion: Anonymous communication from multiparty shuffling protocols. In *NDSS 2022*. The Internet Society, 2022.
26. Michael J. Freedman, Yuval Ishai, Benny Pinkas, and Omer Reingold. Keyword search and oblivious pseudorandom functions. In *TCC 2005*. Springer, 2005.
27. Keith B. Frikken. Privacy-preserving set union. In *Applied Cryptography and Network Security, ACNS 2007*, pages 237–252. Springer, 2007.
28. Jiahui Gao, Son Nguyen, and Ni Trieu. Toward a practical multiparty private set union. Cryptology ePrint Archive, Paper 2023/1930, 2023. Version: 20240316:210303, <https://eprint.iacr.org/archive/2023/1930/20240316:210303>.
29. Gayathri Garimella, Payman Mohassel, Mike Rosulek, Saeed Sadeghian, and Jaspal Singh. Private set operations from oblivious switching. In *PKC 2021*, pages 591–617. Springer, 2021.
30. Gayathri Garimella, Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. Oblivious key-value stores and amplification for private set intersection. In *CRYPTO 2021*, pages 395–425. Springer, 2021.
31. Oded Goldreich. *The Foundations of Cryptography - Volume 2: Basic Applications*. Cambridge University Press, 2004.
32. Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In *Proceedings of the 19th Annual ACM Symposium on Theory of Computing, 1987*, pages 218–229. ACM, 1987.
33. Kyle Hogan, Noah Luther, Nabil Schear, Emily Shen, David Stott, Sophia Yakoubov, and Arkady Yerukhimovich. Secure multiparty computation for cooperative cyber risk assessment. In *IEEE Cybersecurity Development, 2016*. IEEE Computer Society, 2016.
34. Yanxue Jia, Shi-Feng Sun, Hong-Sheng Zhou, Jiajun Du, and Dawu Gu. Shuffle-based private set union: Faster and more secure. In *USENIX 2022*, 2022.
35. Yanxue Jia, Shi-Feng Sun, Hong-Sheng Zhou, and Dawu Gu. Scalable private set union, with stronger security. In *USENIX Security 2024*. USENIX Association, 2024.
36. Marcel Keller, Emmanuela Orsini, and Peter Scholl. MASCOT: faster malicious arithmetic secure computation with oblivious transfer. In *CCS 2016*. ACM, 2016.
37. Lea Kissner and Dawn Xiaodong Song. Privacy-preserving set operations. In *Advances in Cryptology - CRYPTO 2005*, pages 241–257. Springer, 2005.
38. Vladimir Kolesnikov, Ranjit Kumaresan, Mike Rosulek, and Ni Trieu. Efficient batched oblivious PRF with applications to private set intersection. In *CCS 2016*, pages 818–829. ACM, 2016.
39. Vladimir Kolesnikov, Naor Matania, Benny Pinkas, Mike Rosulek, and Ni Trieu. Practical multi-party private set intersection from symmetric-key techniques. In *CCS 2017*, pages 1257–1272. ACM, 2017.
40. Vladimir Kolesnikov, Mike Rosulek, Ni Trieu, and Xiao Wang. Scalable private set union from symmetric-key techniques. In *ASIACRYPT 2019*. Springer, 2019.

41. Arjen K. Lenstra and Tim Voss. Information security risk assessment, aggregation, and mitigation. In *Information Security and Privacy, ACISP 2004*, pages 391–401. Springer, 2004.
42. Xiang Liu and Ying Gao. Scalable multi-party private set union from multi-query secret-shared private membership test. In *ASIACRYPT 2023*. Springer, 2023.
43. Payman Mohassel and Seyed Saeed Sadeghian. How to hide circuits in MPC an efficient framework for private function evaluation. In *EUROCRYPT 2013*, pages 557–574. Springer, 2013.
44. Ofri Nevo, Ni Trieu, and Avishay Yanai. Simple, fast malicious multiparty private set intersection. In *CCS 2021*, pages 1151–1165. ACM, 2021.
45. Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. *Journal of Algorithms*, 51(2):122–144, 2004.
46. Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. Spot-light: Lightweight private set intersection from sparse OT extension. In *CRYPTO 2019*, pages 401–431. Springer, 2019.
47. Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. PSI from PaXoS: Fast, malicious private set intersection. In *EUROCRYPT 2020*. Springer, 2020.
48. Benny Pinkas, Thomas Schneider, Gil Segev, and Michael Zohner. Phasing: Private set intersection using permutation-based hashing. In *USENIX Security 2015*, pages 515–530. USENIX Association, 2015.
49. Benny Pinkas, Thomas Schneider, Oleksandr Tkachenko, and Avishay Yanai. Efficient circuit-based PSI with linear communication. In *EUROCRYPT 2019*, pages 122–153. Springer, 2019.
50. Benny Pinkas, Thomas Schneider, and Michael Zohner. Faster private set intersection based on OT extension. In *USENIX Security 2014*. USENIX Association, 2014.
51. Michael O. Rabin. How to exchange secrets with oblivious transfer. *IACR Cryptol. ePrint Arch.*, page 187, 2005. <http://eprint.iacr.org/2005/187>.
52. Srinivasan Raghuraman and Peter Rindal. Blazing fast PSI from improved OKVS and subfield VOLE. In *CCS 2022*. ACM, 2022.
53. Srinivasan Raghuraman, Peter Rindal, and Titouan Tanguy. Expand-convolute codes for pseudorandom correlation generators from LPN. In *CRYPTO 2023*, pages 602–632. Springer, 2023.
54. Peter Rindal. libOTe: an efficient, portable, and easy to use Oblivious Transfer Library. <https://github.com/osu-crypto/libOTe.git>.
55. Peter Rindal and Phillipp Schoppmann. VOLE-PSI: fast OPRF and circuit-psi from vector-OLE. In *EUROCRYPT 2021*, pages 901–930. Springer, 2021.
56. Lawrence Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In *CRYPTO 2022*. Springer, 2022.
57. Jelle Vos, Mauro Conti, and Zekeriya Erkin. Fast multi-party private set operations in the star topology from secure ands and ors. *IACR Cryptol. ePrint Arch.*, page 721, 2022. <https://eprint.iacr.org/2022/721>.
58. Cong Zhang, Yu Chen, Weiran Liu, Min Zhang, and Dongdai Lin. Optimal private set union from multi-query reverse private membership test. In *USENIX 2023*. USENIX Association, 2023.
59. Shengnan Zhao, Ming Ma, Xiangfu Song, Han Jiang, Yunxue Yan, and Qiuliang Xu. Lightweight threshold private set intersection via oblivious transfer. In *Wireless Algorithms, Systems, and Applications, WASA 2021*. Springer, 2021.

A Leakage Analysis of [28]

The MPSU protocol in [28] is claimed to be secure in the presence of arbitrary colluding participants. However, our analysis would suggest that the protocol fails to achieve this security, and also requires the non-collusion assumption as LG. First, we give a brief review of the protocol.

Apart from MKR-PKE, their protocol utilizes two new ingredients: 1) The conditional oblivious pseudorandom function (cOPRF), an extension they develop on the OPRF, where the sender $\mathcal{S}$ additionally inputs a set Y . If $x \notin Y$, $\mathcal{R}$ receives $F_k(x)$, else $\mathcal{R}$ receives a random value sampled by $\mathcal{S}$. 2) The membership Oblivious Transfer (mOT), where $\mathcal{S}$ inputs an element y and two messages u_0, u_1 , while $\mathcal{R}$ inputs a set X and receives u , one of u_0, u_1 . If $y \in X$, $u = u_0$, else $u = u_1$.

To illustrate insecurity of their protocol, we consider a three-party case where P_1 and P_3 each possess a single item $X_1 = \{x_1\}$ and $X_3 = \{x_3\}$ respectively, while P_2 possesses a set X_2 . We assume that $x_1 = x_3$. According to the protocol (cf. Figure 8 in their paper), in step 3.(a), P_1 and P_2 invoke the OPRF where P_1 acts as $\mathcal{R}$ inputting x_1 and P_2 acts as $\mathcal{S}$ inputting its PRF key k_2 . P_1 receives the PRF value $F_{k_2}(x_1)$. Meanwhile, in step 3.(c), P_2 and P_3 invoke the cOPRF where P_3 acts as $\mathcal{R}$ inputting x_3 , and P_2 acts as $\mathcal{S}$ inputting its PRF key k_2 and the set X_2 . P_3 receives the output w from the cOPRF. By the definition of cOPRF functionality, if $x_3 \notin X_2$, $w = F_{k_2}(x_3)$, otherwise w is a random value.

If P_1 and P_3 collude, they can distinguish the cases where $x_3 \in X_2$ and $x_3 \notin X_2$ by comparing P_1 's output $F_{k_2}(x_1)$ from the OPRF and P_3 's output w from the cOPRF for equality. To elaborate, we recall that $x_1 = x_3$, so $F_{k_2}(x_1) = F_{k_2}(x_3)$. If $F_{k_2}(x_1) = w$, it implies that P_3 receives $F_{k_2}(x_3)$ from the cOPRF, so the coalition learns that $x_3 \notin X_2$; On the contrary, if $F_{k_2}(x_1) \neq w$, it implies that P_3 's output from the cOPRF is not $F_{k_2}(x_3)$, so it is a random value, then the coalition learns that $x_3 \in X_2$. More generally, as long as P_1 and P_3 collude, they can identify whether each element $x \in X_1 \cap X_3$ belongs to X_2 or not, by comparing the PRF value $F_k(x)$ from the OPRF between P_1 and P_2 and the cPRF value (whose condition depends on $x \in X_2$ or not) from the cOPRF between P_2 and P_3 . This acquired knowledge is information leakage in MPSU. Therefore, the non-collusion assumption is required.

B MKR-PKE Appendix

B.1 Indistinguishable Multiple Encryptions

The IND-CPA security of PKE implies security for encryption of multiple messages whose definition is as follows:

Definition 2. A public-key encryption scheme $\mathcal{E} = (\text{Gen}, \text{Enc}, \text{Dec})$ has indistinguishable multiple encryptions if for all PPT adversaries $\mathcal{A}$ s.t. any tuples

$(m_1, \dots, m_q)$ and $(m'_1, \dots, m'_q)$ chosen by $\mathcal{A}$ (where q is polynomial in λ):

$$\begin{aligned} & \{\text{Enc}(pk, m_1), \dots, \text{Enc}(pk, m_q) : (pk, sk) \leftarrow \text{Gen}(1^\lambda)\} \stackrel{c}{\approx} \\ & \{\text{Enc}(pk, m'_1), \dots, \text{Enc}(pk, m'_q) : (pk, sk) \leftarrow \text{Gen}(1^\lambda)\} \end{aligned}$$

B.2 Construction from ElGamal

A MKR-PKE with IND-CPA security can be instantiated with ElGamal encryption as follows:

- The key-generation algorithm Gen takes as input the security parameter 1^λ and generates $(\mathbb{G}, g, p)$, where $\mathbb{G}$ is a cyclic group, g is the generator and q is the order. Outputs sk and $pk = g^{sk}$.
- The randomized encryption algorithm Enc takes as input a public key pk and a plaintext message $x \in \mathbb{G}$, samples $r \leftarrow \mathbb{Z}_q$, and outputs $\text{ct} = (\text{ct}_1, \text{ct}_2) = (g^r, x \cdot pk^r)$.
- The partial decryption algorithm ParDec takes as input a secret key share sk and a ciphertext $\text{ct} = (\text{ct}_1, \text{ct}_2)$, and outputs $\text{ct}' = (\text{ct}_1, \text{ct}_2 \cdot \text{ct}_1^{-sk})$.
- The decryption algorithm Dec takes as input a secret key sk and a ciphertext $\text{ct} = (\text{ct}_1, \text{ct}_2)$, and outputs $x = \text{ct}_2 \cdot \text{ct}_1^{-sk}$.
- The rerandomization algorithm ReRand takes as input pk and a ciphertext $\text{ct} = (\text{ct}_1, \text{ct}_2)$, samples $r \leftarrow \mathbb{Z}_q$, and outputs $\text{ct}' = (\text{ct}_1 \cdot g^{r'}, \text{ct}_2 \cdot pk^{r'})$.

C Multi-Party Private-ID

It is natural to generalize the two-party private-ID [12] to multi-party setting. Suppose there are m parties, each possessing a set of n elements. The multi-party private-ID functionality assigns a unique random identifier to each element across all input sets, ensuring that identical elements in different sets obtain the same identifiers. Each party receives identifiers associated with its own input elements, as well as identifiers associated with the union of all parties' input sets. With multi-party private-ID, the parties can sort their private sets based on a global set of identifiers and perform desired private computations item by item, ensuring alignment of identical elements across their sets. The formal definition of the multi-party private-ID is depicted in Figure 13. We build a concrete multi-party private-ID protocol based on the DDH assumption (described in Figure 14) by extending the “distributed OPRF+PSU” paradigm [20] to multi-party setting.

Theorem 5. *Protocol Π_{MPID} securely implements $\mathcal{F}_{\text{MPID}}$ against any semi-honest adversary corrupting $t < m$ parties in the $\mathcal{F}_{\text{mpsu}}$ -hybrid model, assuming the DDH assumption and $\mathcal{H}$ is a random oracle.*

Correctness. The first two steps essentially realize a multi-party distributed OPRF protocol, where each party P_i inputs a set $\{x_i^1, \dots, x_i^n\}$ and receives its own PRF key k_i along with the PRF values computed on its input set using all

Parameters. m parties $P_1, \dots, P_m$. Size n of input sets. The bit length l of set elements. The range D of identifiers.

Functionality. On input $X_i = \{x_i^1, \dots, x_i^n\} \subseteq \{0, 1\}^l$ from P_i ,

- For every element $x \in \bigcup_{i=1}^m X_i$, choose a random identifier $R(x) \leftarrow D$.
- Define $R^* = \{R(x) | x \in \bigcup_{i=1}^m X_i\}$ and $R_i = \{R(x) | x \in X_i\}$ for $i \in [m]$.
- Give output (R^*, R_i) to P_i .

Fig. 13. Multi-Party Private ID Functionality $\mathcal{F}_{\text{MPID}}$

Parameters. m parties $P_1, \dots, P_m$. Size n of input sets. The bit length l of set elements. A cyclic group $\mathbb{G}$, where g is the generator and q is the order. The identifier range $D = \mathbb{G}$. Hash function $H(x) : \{0, 1\}^l \rightarrow \mathbb{G}$.

Inputs. Each P_i has input $X_i = \{x_i^1, \dots, x_i^n\} \subseteq \{0, 1\}^l$.

Protocol.

1. For $1 \leq i \leq m$, P_i samples $a_i \leftarrow \mathbb{Z}_q$ and $k_i \leftarrow \mathbb{Z}_q$, then sends $\{H(x_i^1)^{a_i}, \dots, H(x_i^n)^{a_i}\}$ to $P_{(i+1) \bmod m}$. For $1 \leq j < m$, $P_{(i+j) \bmod m}$ receives $\{y_i^1, \dots, y_i^n\}$ from $P_{(i+j-1) \bmod m}$. $P_{(i+j) \bmod m}$ computes $\{(y_i^1)^{k_{(i+j) \bmod m}}, \dots, (y_i^n)^{k_{(i+j) \bmod m}}\}$ and sends to $P_{(i+j+1) \bmod m}$.
2. For $1 \leq i \leq m$, P_i receives $H(x_i^j)^{a_i k_{i+1} \dots k_m k_1 \dots k_{i-1}}$, $j \in [n]$, and computes $(H(x_i^j)^{a_i k_{i+1} \dots k_m k_1 \dots k_{i-1}})^{-a_i k_i} = H(x_i^j)^{k_1 \dots k_m}$. We denote $\{H(x_i^1)^{k_1 \dots k_m}, \dots, H(x_i^n)^{k_1 \dots k_m}\}$ as $R_i = \{r_i^1, \dots, r_i^n\}$, where each $r_i^j \in \mathbb{G}$.
3. The parties invoke $\mathcal{F}_{\text{mpsu}}$ where P_i inputs $R_i = \{r_i^1, \dots, r_i^n\}$. P_1 receives the union $R^* = \bigcup_{i=1}^m R_i$, and sends it to other parties.
4. Each party P_i outputs (R^*, R_i) .

Fig. 14. DH-based Multi-Party Private ID $\mathcal{H}_{\text{MPID}}$

parties' keys $k_1, \dots, k_m$, denoted as $\{\text{PRF}_{k_1, \dots, k_m}(x_i^1), \dots, \text{PRF}_{k_1, \dots, k_m}(x_i^n)\}$. In Figure 14, each party P_i first masks the hash value of each element by raising it to the power of a_i . Then these masked values are then cycled among the parties, with each party raising them to the power of its key. Finally, P_i unmask the values and raises them to the power of its won key k_i , resulting in the distributed PRF values $\text{PRF}_{k_1, \dots, k_m}(x) = H(x)^{k_1 \cdots k_m}$. Even if $m - 1$ parties collude, there still exists one exponent private to the adversary, ensuring that the PRF values remain pseudorandom according to the DDH assumption.

In the step 3, the parties invoke an MPSU protocol on their distributed PRF values, so that they can securely receive all PRF values on the union of input sets.

Security. Assuming H is a random oracle, the security of protocol follows immediately from the DDH assumption and the security of our PK-MPSU.

Modern DH-based cryptosystems often use elliptic curves for the underlying cyclic group, due to their compact size and better efficiency at a given level of security. In this context, the first two steps of the protocol have relatively low communication costs, meanwhile, the element space of the MPU protocol in step 3 is EC points, which aligns perfectly with the element space of our PK-MPSU. This alignment allows us to instantiate the MPSU functionality using our PK-MPSU, leveraging its lowest communication costs. As a result, the entire multi-party private-ID protocol achieves extremely low communication costs, making it well-suited for deployment in bandwidth-restricted environments.

Another benefit of instantiating the MPSU functionality with our PK-MPSU is to achieve linear computation and communication complexity. Since the multi-party distributed OPRF phase has linear computation and communication, by integrating our PK-MPSU, the entire multi-party private-ID protocol can maintain the linear complexities throughout.